

Instituto Politécnico do Porto

Instituto Superior de Engenharia do Porto



XD Fusion — Arquitetura Event-Driven e Integração com Sistemas

XD Software

2025/2026

Daniel Filipe Flores Novo (1231699)

Instituto Politécnico do Porto

Instituto Superior de Engenharia do Porto



XD Fusion — Arquitetura Event-Driven e Integração com Sistemas

XD Software

2025/2026

Daniel Filipe Flores Novo (1231699)

Orientador: **Agostinho Gil Teixeira Lopes (AGL)**

Supervisor: **Ricardo Torres Varzim Faria**

LEI - Licenciatura em Engenharia Informática

Junho, 2026

ISEP INSTITUTO SUPERIOR
DE ENGENHARIA DO PORTO

“Any sufficiently advanced technology is indistinguishable from magic.”

Arthur C. Clarke

Agradecimentos

A conclusão desta etapa académica representa muito mais do que o culminar de um plano de estudos ou a entrega de um projeto técnico. É o reflexo de anos de persistência, de noites em claro e de uma transformação profunda na forma como encaro o futuro. Por essa razão, é com enorme orgulho e justa gratidão que dedico este espaço àqueles que caminharam ao meu lado.

À minha família, em especial aos meus pais. Sei bem que o caminho até este dia não foi feito sem sacrifícios, sem preocupações e sem uma entrega da vossa parte para que nada me faltasse. O vosso apoio incondicional foi o porto seguro para onde pude sempre regressar sempre que a pressão e a exaustão pesaram. Esta vitória não é apenas minha; é vossa, fruto do vosso amor e da vossa crença inabalável nas minhas capacidades, mesmo quando eu próprio duvidava delas.

À minha namorada, que foi um dos pilares mais sólido e a luz mais brilhante nos momentos mais exigentes desta jornada. Obrigado por seres a minha maior conselheira e amiga, pela paciência sem limites nas fases de maior stress e por me recordares constantemente de quem eu sou e do que sou capaz de criar. A tua presença ao meu lado transformou os dias cinzentos, ensinando-me a respeitar o meu próprio ritmo. Este trabalho carrega também um pedaço da tua cooperação, paciência e esforço.

Aos meus colegas e amigos de curso, em particular ao grupo que se tornou a minha segunda família dentro das paredes do ISEP, os Lobos do Mar, Diogo, Rui e Henrique. Partilhámos a dureza das entregas, a fadiga dos exames, no entanto também os momentos bons e brincadeiras dentro e fora do ambiente de trabalho do ISEP. O espírito de entreajuda, a camaradagem e as gargalhadas nos momentos de maior aperto foram o combustível necessário para não baixar os braços.

Ao meu orientador, o Professor Agostinho Gil Teixeira Lopes, pelo voto de confiança ao aceitar a orientação deste projeto e pela disponibilidade constante. O seu rigor e cooperação foram fundamentais para que este trabalho ganhasse forma e atingisse os objetivos propostos.

À XD Software, por me ter aberto as portas do mundo profissional e por me ter acolhido com tanta confiança no desenvolvimento do projeto XD Fusion. Um agradecimento muito especial ao meu supervisor de estágio, o Eng. Ricardo Varzim, cuja mentoria e paciência foram essenciais. Mais do que orientar o desenvolvimento técnico o seu apoio e o ambiente

tranquilo e amigável com que me acolheu, ajudaram-me a moldar a minha postura e a ter uma entrada mais agradável neste mundo profissional, assim crescendo como futuro engenheiro.

Por fim, olho para trás e sinto um orgulho imenso no caminho percorrido. Este relatório prova que a Engenharia informática vai muito além dos sistemas e das arquiteturas complexas que aqui idealizámos; ela testa os nossos limites, obriga-nos a gerir a frustração e a evoluir. Entro nesta nova fase da minha vida com a certeza de que cada queda foi uma lição e de que estou pronto para o que vier.

A todos os que fizeram parte desta história, o meu eterno e sincero agradecimento.

Resumo

O presente relatório descreve o trabalho realizado no estágio curricular desenvolvido na XD Software, no âmbito do projeto XD Fusion. Os sistemas ERP e POS da organização dependem fortemente de intervenção humana para reagir a determinadas situações de negócio, o que limita a eficiência operacional e atrasa a resposta a eventos relevantes.

Para colmatar esta lacuna, foi desenvolvida uma solução de automação baseada numa arquitetura orientada a eventos, capaz de processar ocorrências em tempo real e executar automaticamente fluxos de trabalho associados.

A solução implementada assenta num pipeline de eventos suportado pelo Redis, que integra um fluxo reativo, acionado por eventos imediatos como o registo de novos clientes, e um fluxo periódico, responsável pela deteção de inatividade comercial.

A plataforma n8n foi utilizada para orquestrar os *workflows* de automação, com integração à API REST do sistema via JWT.

Foram concretizados quatro casos de uso: notificação do gestor comercial, envio de mensagem de boas-vindas por e-mail ou WhatsApp, alerta de inatividade e sincronização com a plataforma Brevo.

A qualidade da solução foi assegurada através de testes unitários, de integração e funcionais, executados automaticamente em cada alteração ao repositório.

Os objetivos definidos foram integralmente cumpridos, tendo a solução demonstrado eficácia em ambiente controlado e potencial de extensão a novos cenários de automação no ecossistema da XD Software.

Palavras-chave (Tema): Arquitetura orientada a eventos, Automação de processos, Integração de sistemas, Gestão do relacionamento com clientes.

Palavras-chave (Tecnologias): .NET 8, Redis Pub/Sub, n8n, *Webhooks*, JSON Web Tokens (JWT).

Índice

1	<i>Introdução</i>	1
1.1	Contexto	1
1.2	Descrição do Problema	1
1.2.1	Objetivos	2
1.2.2	Abordagem	3
1.2.3	Contributos	3
1.2.4	Planeamento do trabalho	4
1.3	Estrutura do relatório	5
2	<i>Estado da arte</i>	6
2.1	Trabalhos relacionados	6
2.1.1	Evolução dos sistemas de gestão empresarial	6
2.1.2	Gestão de informação em sistemas empresariais	8
2.1.3	Automação em Sistemas CRM	8
2.2	Tecnologias existentes	9
2.2.1	API REST	9
2.2.2	Webhooks	11
2.2.2.1	Polling x Webhooks	11
2.2.3	Plataformas de automação de workflows	12
2.3	Contexto tecnológico	13
2.3.1	Infraestrutura tecnológica utilizada na XD Software	13
2.3.2	Tecnologias e padrões de comunicação	14
3	<i>Análise e desenho da solução</i>	16
3.1	Domínio do problema	16
3.2	Requisitos funcionais e não funcionais	18
3.2.1	Requisitos funcionais	18
3.2.2	Requisitos não funcionais	19
3.3	Use Cases	Erro! Marcador não definido.
3.3.1	Webhook <i>customer.registered</i>	21
		22
3.3.1.1	UC-01: Notificar o gestor comercial sobre registo de um novo cliente	22
3.3.1.2	UC-02: Enviar mensagem de boas-vindas a cliente por e-mail ou WhatsApp	23
3.3.1.3	UC-04: Sincronizar novo cliente com plataforma de marketing	24

3.3.2	<i>Webhook customer.inactive.detected</i>	25
3.3.2.1	UC-03: Alertar gestor comercial de clientes sem atividade comercial recente ..	26
3.4	Desenho	27
3.4.1	Vista lógica	27
3.4.2	Vista de implementação	29
3.4.3	Vista física	31
3.4.4	Vista de processos	31
4	Implementação da Solução	34
4.1	Descrição da implementação	34
4.1.1	Tecnologias e metodologias utilizadas	34
4.1.2	Implementação da arquitetura orientada a eventos	35
4.1.2.1	Fluxo reativo	36
4.1.2.2	Fluxo periódico	38
4.1.3	Casos de uso e fluxos de negócio	41
4.1.3.1	UC-01: Notificar o gestor comercial sobre um novo cliente	41
4.1.3.2	UC-02: Enviar mensagem de boas-vindas ao novo cliente	44
4.1.3.3	UC-03: Alertar o gestor sobre clientes com inatividade comercial	47
4.1.3.4	UC-04: Sincronizar o novo cliente com a plataforma de marketing Brevo	49
4.2	Testes	50
4.2.1	Abordagem de verificação	50
4.2.2	Verificação dos componentes de domínio e aplicação	51
4.2.3	Verificação dos fluxos de execução em segundo plano	53
4.2.4	Verificação da camada de interface	55
4.2.5	Resultados e integração contínua	56
4.3	Avaliação da solução	56
4.3.1	Cumprimento dos requisitos funcionais	56
4.3.2	Cumprimento dos requisitos não funcionais	57
4.3.3	Discussão	58
5	Conclusões	60
5.1	Objetivos concretizados	60
5.2	Limitações e trabalho futuro	62
5.3	Apreciação final	63
	Referências	65

Índice de Figuras

Figura 1 - Evolução dos sistemas de gestão empresarial (Vockan, 2024)	7
Figura 2 - Comunicação através de uma API REST (Venelouis, 2024)	10
Figura 3 - Polling x Webhooks	12
Figura 4 - Domain Model	17
Figura 5 - Use Case Diagram	21
Figura 6 - UC-01: Vista de processos (Nível 1)	22
Figura 7 - UC-02: Vista de processos (Nível 1)	23
Figura 8 - UC-04: Vista de processos (Nível 1)	24
Figura 9 - UC-03: Vista de processos (Nível 1)	26
Figura 10 - Vista Lógica (Nível 1)	27
Figura 11 - Vista lógica (Nível 2)	28
Figura 12 - Vista Lógica (Nível 3)	29
Figura 13 - Vista de Implementação (Nível 2)	30
Figura 14 - Vista de Implementação (Nível 3)	30
Figura 15 - Vista Física do Sistema	31
Figura 16 - UC-01-02-04: Vista de Processos (Nível 3)	32
Figura 17 - UC-03: Vista de Processos (Nível 3)	33
Figura 18 - Pipeline de publicação, processamento e entrega de eventos reativos	37
Figura 19 - Fluxo periódico de deteção e publicação de clientes inativos	39
Figura 20 - UC-01: fluxo n8n	42
Figura 21 - E-mail gestor de novo cliente	43
Figura 22 - UC-02: fluxo n8n e-mail	44
Figura 23 - E-mail boas-vindas	45
Figura 24 - UC-02: fluxo n8n whatsapp	46
Figura 25 - Whatsapp boas-vindas	46

Figura 26 - UC-02: fluxo n8n vazio.....	46
Figura 27 - UC-03: fluxo n8n.....	47
Figura 28 - Mensagem inatividade	48
Figura 29 - UC-04: fluxo n8n	49
Figura 30 - Lista de clientes Brevo	50
Figura 31 - Run testes (succeeded)	Erro! Marcador não definido.

Índice de Tabelas

Tabela 1 - Exemplos de automação em sistemas CRM	13
Tabela 2 - Categorias do modelo FURPS+	18
Tabela 3 - Requisitos funcionais	19
Tabela 4 - Requisitos não funcionais	20
Tabela 5 - Regras de avaliação de inatividade comercial	40
Tabela 6 - Avaliação dos requisitos funcionais.....	56
Tabela 7 - Avaliação dos requisitos não funcionais.....	57
Tabela 8 - Matriz de análise da concretização dos objetivos pretendidos	61

Índice de Listagens

Listagem 1 - Payload do webhook customer.registered	22
Listagem 2 – Parte do payload do webhook customer.inactivity.detected	25
Listagem 3 - RedisEventSubscriber: isolamento de contexto por mensagem via CreateScope	37
Listagem 4 - WeeklyPeriodicScheduleModePlugin: avaliação da subscrição como devida pelo método IsDue	38
Listagem 5 - CustomerInactiveDetectionQuery: normalização Unicode do distrito para correspondência com zonas de venda	41
Listagem 6 - Classe CustomerInactivityEvaluatorTests	52
Listagem 7 - Classe CustomerInactivityDetectedPeriodicWebhookHandlerTests	52
Listagem 8 - Classe WebhookEventPipelineTests	53
Listagem 9 - Classe WeeklyPeriodicScheduleModePluginTests	54
Listagem 10 - Classe PeriodicWebhookWorkerTests	54
Listagem 11 - Classe WebhookDeliveryWorkerTests	54
Listagem 12 - Classe WebhookSubscriptionsControllerTests	55

Notação e Glossário

API	Interface de Programação de Aplicações, conjunto de mecanismos que permite a comunicação e troca de dados entre sistemas.
CI/CD	<i>Continuous Integration/Continuous Deployment</i> , conjunto de práticas de automação que permitem integrar, testar e disponibilizar software de forma contínua.
CQRS	Segregação de Responsabilidade de Comando e Consulta (<i>Command Query Responsibility Segregation</i>), padrão arquitetural que separa as operações de leitura das operações de escrita num sistema.
CRM	<i>Customer Relationship Management</i> , sistema de gestão do relacionamento com clientes.
EF Core	Mapeador objeto-relacional nativo da plataforma .NET (<i>Entity Framework Core</i>), utilizado para abstrair o acesso à base de dados.
ERP	<i>Enterprise Resource Planning</i> , sistema integrado de gestão empresarial que centraliza processos como vendas, inventário e faturação.
FURPS+	Modelo de classificação de requisitos que abrange Funcionalidade, Usabilidade, Fiabilidade (<i>Reliability</i>), Desempenho (<i>Performance</i>) e Suportabilidade, com extensões para requisitos adicionais.
HMAC	<i>Hash-based Message Authentication Code</i> , mecanismo criptográfico utilizado para verificar a autenticidade e integridade das mensagens transmitidas entre sistemas.
HTTP	Protocolo de Transferência de Hipertexto (<i>Hypertext Transfer Protocol</i>), protocolo de comunicação utilizado para a transferência de dados na web.
IoC	Inversão de Controlo (<i>Inversion of Control</i>), princípio de design em que a gestão de dependências é delegada a um contentor externo em vez de ser controlada pelo próprio código.
ISEP	Instituto Superior de Engenharia do Porto

JSON	JavaScript <i>Object Notation</i> , formato leve de troca de dados utilizado em API e sistemas web.
JWT	JSON <i>Web Token</i> , mecanismo de autenticação baseado em tokens utilizado para garantir a segurança das comunicações entre sistemas.
LEI	Licenciatura em Engenharia Informática
MRP	<i>Material Requirements Planning</i> , sistema de planeamento das necessidades de materiais em processos de produção.
MRP II	<i>Manufacturing Resource Planning</i> , evolução do MRP que integra planeamento da produção com outros recursos empresariais.
POS	<i>Point of Sale</i> , sistema de ponto de venda utilizado para registo e processamento de transações comerciais.
PROJ	Unidade Curricular de Projeto
Redis	Servidor de Dicionário Remoto (<i>Remote Dictionary Server</i>), sistema de armazenamento de dados em memória utilizado como mecanismo de publicação/subscrição de mensagens e suporte a bloqueios distribuídos.
REST	<i>Representational State Transfer</i> , estilo arquitetural utilizado no desenvolvimento de API e serviços web.
TCP	Protocolo de Controlo de Transmissão (<i>Transmission Control Protocol</i>), protocolo de comunicação que garante a entrega ordenada e fiável de dados entre sistemas.
UC	<i>Use Case</i> , caso de uso que descreve uma interação entre o sistema e os seus atores para atingir um determinado objetivo.
URL	Localizador Uniforme de Recursos (<i>Uniform Resource Locator</i>), endereço que identifica de forma única um recurso na internet ou numa rede.

1 Introdução

O presente capítulo tem como objetivo contextualizar o trabalho desenvolvido ao longo do estágio curricular, apresentando o enquadramento institucional e empresarial no qual este se insere, bem como a descrição do problema que motivou o projeto.

São ainda apresentados os objetivos definidos, a abordagem adotada para a sua resolução, os contributos esperados para a organização e o planeamento seguido ao longo do período de estágio. Por fim, descreve-se a estrutura do presente relatório.

1.1 Contexto

O documento foi desenvolvido no âmbito da unidade curricular direcionada à realização do Projeto (LEI-PROJ), incluída no plano de estudo do segundo semestre do terceiro ano do curso de Licenciatura em Engenharia Informática do Instituto Superior de Engenharia do Porto (ISEP), com o objetivo de proporcionar aos alunos envolvidos, uma experiência profissional mais autêntica, assim promovendo o seu desenvolvimento profissional.

Este estágio foi realizado na empresa XD Software, sendo esta uma organização especializada no desenvolvimento de soluções tecnológicas para gestão empresarial, principalmente ERP (*Enterprise Resource Planning*) e POS (*Point of Sale*).

No âmbito deste estágio, a XD Software propôs o desenvolvimento do projeto XD Fusion, uma iniciativa que visa a automação de processos empresariais e a integração inteligente de sistemas através de uma arquitetura orientada a eventos. Este projeto enquadra-se na necessidade crescente de as organizações evoluírem de modelos reativos e manuais para sistemas capazes de responder automaticamente a eventos em tempo real.

1.2 Descrição do Problema

Apesar de serem muito vantajosos e de toda a sua robustez, o funcionamento dos ERP e POS é, na grande maioria dos casos, bastante dependente da consulta manual de informação, isto pois os utilizadores necessitam de verificar essas informações de um modo não automatizado, sendo acedendo a relatórios, analisando dados, ou até mesmo executando ações manualmente. Esta necessidade da envolvimento humana promove a limitação da eficiência operacional, bem como da resposta em tempo real.

Estes modelos de resposta reativa, que são normalmente suportados por sistemas de *polling* e sequencialmente consultas a bases de dados e API, consome recursos em excesso e pode atrasar reações, não sendo assim em tempo real, entre outras desvantagens que dificultam a implementação de um sistema de processos automáticos e a atuação imediata perante certos eventos.

Com isto em mente, foi identificada a necessidade de evoluir a arquitetura para uma arquitetura orientada a eventos, capaz de usar *webhooks*, permitindo acesso imediato e automático a dados e à execução de ações.

1.2.1 Objetivos

Em um ponto de vista mais abrangente, que já foi abordado anteriormente, o objetivo do estágio é ser capaz de contribuir para a evolução da plataforma XD Fusion, utilizando para isso mecanismos de integração e automação de processos e respostas, através de uma arquitetura de eventos.

Para atingir tal objetivo, foram definidos certos objetivos mais específicos, sendo estes:

- **Desenvolvimento de templates de automação** capazes de automatizar ações tendo por base eventos do sistema. Estes *workflows* irão permitir a existência de respostas automáticas a necessidades de negócio ou de qualquer tipo pretendido.
- **Integração com a API REST do ERP e POS**, implementando mecanismos de comunicação com a API REST, tendo incluído a autenticação com JWT, bem como o consumo de *endpoints*, o que permite o acesso e tratamento de dados em tempo real.
- **Adoção de arquitetura *event-driven***, implementando *webhooks* para a receção de eventos em tempo real, assim promovendo a transição desde um modelo baseado em consultas exigindo ação manual, para um modelo orientado a eventos, permitindo este, processos automáticos.
- **Automação de processos empresariais**, e de tarefas repetitivas, bem como a execução automática de ações desencadeadas por certos eventos, assim reduzindo a intervenção manual e aumentando a eficiência operacional.
- **Testes e validação das soluções desenvolvidas**, realizada e garantida através de testes funcionais, de testes de integração, e ainda através de testes de desempenho. Os primeiros com o objetivo de simular eventos reais do sistema, e o último para avaliar a robustez da solução desenvolvida. Com estes, o objetivo é obter a melhor solução possível.

- **Documentação técnica**, produzida com vista nas integrações e *workflows* que foram desenvolvidos, assim assegurando que a sua utilização terá o suporte necessário para o seu correto funcionamento, bem como a sua manutenção futura.

1.2.2 Abordagem

A abordagem adotada para a resolução do problema baseia-se na conceção de uma arquitetura de integração orientada a eventos, suportada por mecanismos de comunicação em tempo real e integração com sistemas empresariais.

Numa fase inicial, será explorada a API REST do sistema ERP e POS, permitindo compreender a estrutura dos dados e implementar mecanismos de autenticação baseados em *tokens* JWT. Esta integração possibilitará o acesso estruturado a informação operacional necessária aos processos de automação.

Posteriormente, será implementada uma arquitetura *event-driven*, baseada na receção de *webhooks* provenientes do sistema XD Cloud. Estes eventos serão processados através de uma camada de orquestração responsável por encaminhar a informação para os respetivos *workflows* de automação, permitindo a execução automática de ações com base em ocorrências de negócio.

A implementação das automações será suportada por *templates* reutilizáveis, concebidos para responder a diferentes cenários operacionais e possibilitar a integração com serviços externos sempre que necessário.

Ao nível da organização do trabalho, e apesar de a entidade acolhedora não adotar formalmente a metodologia *Scrum* nem uma gestão rígida de prazos, optou-se por manter uma estrutura de desenvolvimento baseada em princípios ágeis. Esta decisão resulta da familiaridade adquirida em contexto académico, permitindo organizar tarefas, definir objetivos por ciclos de trabalho e acompanhar a evolução do projeto de forma estruturada.

Deste modo, o desenvolvimento será orientado por fases iterativas, com definição progressiva de objetivos e monitorização contínua das atividades realizadas, promovendo flexibilidade e adaptação às necessidades do projeto.

1.2.3 Contributos

Ao concluir este projeto, a empresa procura adquirir contributos significativos a nível técnico e estratégico, destacando-se os principais aspetos inovadores e benéficos:

- **Transição para um Modelo Reativo:** Evolução de uma gestão manual para uma arquitetura orientada a eventos, substituindo processos de consulta periódica por reações automáticas e instantâneas a eventos de negócio registrados no ERP.
- **Otimização de Recursos Humanos:** Automação de tarefas redundantes, reduzindo drasticamente o erro humano e a carga administrativa, assim libertando colaboradores para funções de valor acrescentado.
- **Inteligência de Negócio em Tempo Real:** Capacidade de gerar e encaminhar informação crítica sobre clientes no momento exato em que os dados são produzidos, acelerando significativamente o ciclo de tomada de decisão comercial.
- **Gestão Automatizada do Relacionamento com Clientes:** Estabelecimento de fluxos automáticos de acompanhamento, desde a receção de um novo cliente até ao alerta ao gestor responsável por clientes inativos, resultando numa resposta comercial mais rápida, consistente e personalizada.

1.2.4 Planeamento do trabalho

O desenvolvimento do projeto foi organizado em quatro fases principais, de modo a garantir um progresso estável e controlado ao longo do período de estágio.

- Na primeira fase, entre 23 de fevereiro e 15 de março, realizou-se a fundação técnica da solução, abrangendo a configuração do ambiente de desenvolvimento, a exploração da API REST do sistema ERP e a implementação da autenticação por *tokens JWT*.
- Na segunda fase, de 16 de março a 12 de abril, foi implementada a arquitetura orientada a eventos, com a criação do mecanismo de receção e publicação de webhooks em tempo real e a validação da segurança das comunicações.
- Na terceira fase, entre 13 de abril e 10 de maio, foram desenvolvidas as automações de CRM: notificação ao gestor comercial na criação de um novo cliente, envio de e-mail de boas-vindas ao cliente, sincronização com a plataforma Brevo e deteção e alerta de clientes inativos.
- Na última fase, entre 11 e 25 de maio, a solução foi validada através de testes, documentação técnica e preparação da apresentação final.

1.3 Estrutura do relatório

O presente relatório encontra-se dividido em cinco capítulos que se distinguem entre si através do conteúdo que abordam. Estes descrevem e detalham o percurso previsto para o desenvolvimento da solução pretendida, bem como os resultados obtidos ao longo do caminho.

- **Capítulo 1**, o presente capítulo, fornece a introdução e o contexto do projeto, apresentando também a motivação, os objetivos propostos, a abordagem realizada e os contributos esperados para a organização.
- **Capítulo 2**, apresenta o estado da arte e toda a fundamentação teórica necessária para a compreensão do projeto. Explorando, assim, vários tipos de arquiteturas orientadas a eventos existentes no mercado, juntamente com as suas vantagens e desvantagens individuais.
- **Capítulo 3**, demonstra a análise e o *design* da solução, utilizando modelos e diagramas para ilustrá-la e fundamentá-la.
- **Capítulo 4**, descreve o processo de desenvolvimento da solução proposta, bem como os testes aplicados para validar a robustez do sistema.
- Finalmente, o **Capítulo 5**, contém uma síntese das tarefas realizadas, relacionando os resultados obtidos com os objetivos previamente estabelecidos. Inclui ainda uma reflexão final sobre o impacto do projeto e possíveis melhorias futuras.

2 Estado da arte

À medida que os anos passaram, as organizações sentiram a necessidade de evoluir desde sistemas baseados em processos manuais e consultas periódicas para sistemas reativos, capazes de responder automaticamente a eventos. Entre as tecnologias que viabilizam essa transição destacam-se as API REST, os *webhooks* e as plataformas de gestão de *workflows*, como o n8n (Lokesh Gupta, 2025; Martin Fowler, 2017).

2.1 Trabalhos relacionados

A presente secção apresenta uma revisão dos trabalhos e conceitos relacionados com o tema em estudo. Analisa-se a evolução histórica dos sistemas de gestão empresarial, a transformação dos modelos de gestão de informação em contexto organizacional e os mecanismos de automação aplicados a sistemas de CRM.

2.1.1 Evolução dos sistemas de gestão empresarial

Como referido anteriormente, os sistemas de gestão empresarial evoluíram significativamente ao longo das últimas décadas, evolução essa que pode ser dividida nas seguintes fases principais (Oracle, 2025):

1. Décadas de 1960 e 1970 – MRP (*Material Requirements Planning*)

No início, estes primeiros sistemas apareceram com o desenvolvimento do MRP, e tinham como objetivo principal o apoio ao planeamento da produção e ao controlo de inventário.

Este permitiu melhorar a eficiência da gestão de stocks e reduzir desperdícios nos processos produtivos

2. Década de 1980 – MRP II (*Manufacturing Resource Planning*)

De seguida, graças à evolução da tecnologia, o MRP evoluiu também, para o MRP II, expandindo e melhorando as funcionalidades e capacidades previamente possuídas, possibilitando agora a inclusão do planeamento da capacidade produtiva, da gestão financeira e de outros recursos das organizações. Assim, tornou-se possível obter uma visão mais abrangente das operações empresariais.

3. Década de 1990 – ERP (*Enterprise Resource Planning*)

Nesta década surgiu o ERP, o qual representa um marco importante nesta evolução dos sistemas de gestão empresarial, pois permitem integrar diferentes departamentos numa única plataforma, facilitando assim, a partilha de informações e conhecimentos entre áreas anteriormente afastadas como contabilidade e recursos humanos.

4. Anos 2000 – Integração através da internet

Com a, ainda mais recente, expansão da internet e de outras tecnologias relacionadas, os sistemas ERP passaram a habilitar a integração com aplicações externas e serviços online, o que facilitou ainda mais a comunicação entre os diferentes setores empresariais.

5. Atualidade – ERP *cloud* e sistemas inteligentes

Hoje em dia, muitos dos sistemas de gestão empresarial, são disponibilizados através de plataformas *cloud*, o que permite uma maior flexibilidade, escalabilidade e acesso remoto aos dados.

Além disso, estes sistemas atuais, frequentemente contém funcionalidades de análise de dados e integração com outras ferramentas digitais, permitindo às organizações melhorar a tomada de decisão e a eficiência operacional.

A Figura 1 ilustra a evolução dos sistemas de gestão empresarial ao longo das décadas.



Figura 1 - Evolução dos sistemas de gestão empresarial (Vockan, 2024)

2.1.2 Gestão de informação em sistemas empresariais

Por muito tempo, os sistemas empresariais foram vistos principalmente como ferramentas cujo objetivo era o armazenamento e consulta de informações, ou seja, limitavam-se principalmente a registrar dados dos clientes, das vendas, ou das operações internas, permitindo a posterior análise dos mesmos por parte dos utilizadores.

Nesse mesmo modelo tradicional, o sistema é essencialmente um repositório de informação, onde os dados são introduzidos pelos utilizadores manualmente, e quando necessário, consultados para apoiar decisões ou executar certas tarefas, e embora esta abordagem tivesse permitido a melhoria significativa da organização de informações das empresas, atualmente não é um método viável, visto que apresenta limitações, principalmente quando o objetivo é fornecer uma resposta rápida a eventos relevantes.

Entre essas limitações, encontra-se o facto de, como referido anteriormente, muitas das ações envolvidas serem dependentes da intervenção humana. Por exemplo, no caso de um novo cliente ser registado no sistema, um colaborador pode ser obrigado a realizar uma verificação manual das respetivas informações, e possivelmente realizar tarefas consequentes, como contactar esse novo cliente, chegando à conclusão de que todas estas intervenções e impasses intermédios resultam em atrasos, para além de aumentar a probabilidade da ocorrência de falhas.

Este problema, ganhou muito peso com a evolução das tecnologias digitais, criando a necessidade do desenvolvimento de sistemas com a capacidade de reagir automaticamente a determinadas ocorrências, ou seja, implicou a que fossem adotados novos modelos, modelos estes baseados em eventos onde, com a ocorrência de algum acontecimento relevante no sistema, as ações de resposta são desencadeadas automaticamente, e, em um sistema orientado a eventos, essa criação de um novo cliente, alteração de algum dado ou ausência prolongada de atividade, podem ser interpretados como eventos e desencadeariam automaticamente uma resposta. (Hohpe, 2006)

A adoção desta abordagem, possibilita a redução de dependência indesejada de processos manuais e a melhoria da capacidade de fornecer respostas rápidas.

2.1.3 Automação em Sistemas CRM

Atualmente, a gestão de clientes assume um papel importante no funcionamento das empresas. Os sistemas de *Customer Relationship Management* (CRM) são utilizados para

armazenar e gerir informações relacionadas aos clientes, tais como contactos, histórico de compras ou interações realizadas com a empresa. Desta forma, as organizações conseguem obter uma visão mais completa sobre os seus clientes, facilitando os processos de análise e a tomada de decisões estratégicas (Salesforce, 2024).

Com o aumento do volume de dados e interações registadas nestes sistemas, tornou-se cada vez mais relevante a automação de determinados processos associados à gestão de clientes. A automação em sistemas CRM consiste na execução automática de determinadas ações quando ocorre um evento específico relacionado com um cliente ou contacto.

Estas automações permitem reagir rapidamente a acontecimentos relevantes, melhorando a eficiência dos processos internos e a qualidade da relação com o cliente. Por exemplo, quando um novo cliente é registado no sistema, pode ser automaticamente enviada uma mensagem de boas-vindas ou gerada uma notificação para a equipa comercial responsável.

Outro aspeto importante da automação em CRM é a possibilidade de integração com outras plataformas e serviços digitais. A informação relativa a novos clientes pode, por exemplo, ser automaticamente enviada para ferramentas de comunicação ou plataformas de marketing digital, permitindo manter bases de dados atualizadas e facilitar a execução de campanhas ou contactos comerciais.

Deste modo, estes mecanismos de automação permitem reduzir a necessidade de intervenção manual em tarefas repetitivas, diminuir a probabilidade de erros humanos e garantir que determinadas ações são executadas de forma consistente sempre que ocorrem eventos relevantes no sistema.

2.2 Tecnologias existentes

Nesta secção descrevem-se as principais tecnologias e padrões que suportam a solução desenvolvida, nomeadamente as API REST como mecanismo de acesso a dados, os *webhooks* como canal de comunicação orientado a eventos e as plataformas de orquestração de *workflows* como meio de automação de processos.

2.2.1 API REST

As API, ou, *Application Programming Interfaces* são um dos principais mecanismos que permitem a comunicação entre diferentes sistemas informático, sendo através destas que

as aplicações podem aceder a funcionalidades ou dados disponibilizados por outra aplicação, permitindo a integração entre sistemas distintos.

Entre os diferentes tipos de API existentes, as REST API (Representational State Transfer) são dos padrões mais utilizados na comunicação entre sistemas empresariais e serviços web. Este modelo baseia-se na utilização do protocolo HTTP, permitindo que diferentes aplicações comuniquem entre si através de pedidos e respostas estruturados, permite que a troca de dados ocorra de forma padronizada e eficiente, facilitando a integração entre sistemas empresariais, plataformas web e serviços externos (Mozilla Developer Network, 2024).

Numa API REST, os recursos do sistema são normalmente representados através de URLs, sendo possível realizar diferentes operações sobre esses recursos através de métodos HTTP específicos. Entre os métodos mais utilizados encontram-se:

- **GET**, utilizado para obter informações;
- **POST**, utilizado para criar novos dados;
- **PUT ou PATCH**, utilizados para atualizar dados existentes;
- **DELETE**, utilizado para remover dados.

No contexto de sistemas empresariais, as API REST são frequentemente utilizadas para permitir que aplicações externas acessem a dados de sistemas ERP ou CRM, possibilitando a criação de integrações automáticas e a implementação de processos de automação baseados em eventos.

A Figura 2 ilustra o modelo de comunicação entre sistemas através de uma API REST.

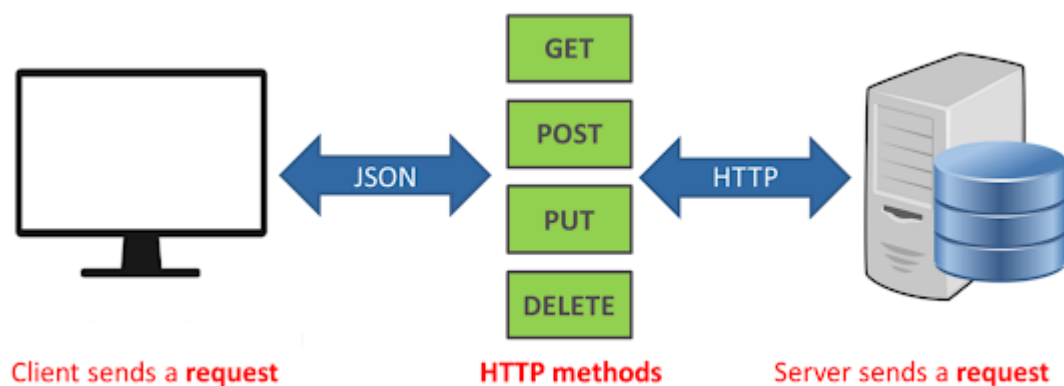


Figura 2 - Comunicação através de uma API REST (Venelouis, 2024)

2.2.2 Webhooks

O uso de *webhooks* é vantajoso no sentido de permitir a automação da comunicação entre aplicações diferentes através da internet, e, ao contrário de métodos tradicionais de consulta periódica de informação nos quais uma aplicação necessita de verificar constantemente se existem novos dados disponíveis, estes permitem que um sistema envie automaticamente uma notificação para outro sistema sempre que ocorre um determinado evento (Matthew Guay, 2025).

Estes funcionam através do envio de um pedido HTTP para um endereço específico, normalmente designado por *endpoint*, sempre que um evento previamente definido é detetado. Esta contém, normalmente, dados estruturados em formato JSON, permitindo que o sistema que recebe o pedido processe a informação recebida e realize as ações pretendidas.

Resumindo, os *webhooks* permitem implementar mecanismos de comunicação em tempo real entre aplicações, reduzindo a necessidade de consultas constantes aos sistemas.

2.2.2.1 Polling x Webhooks

Antes da utilização generalizada de *webhooks*, muitas das integrações eram estabelecidas através de *polling*. Neste, uma aplicação realiza pedidos periódicos a outra aplicação para verificar se existem novos dados disponíveis, ou seja, este processo não é otimizado, visto que causa um consumo desnecessário de recursos, uma vez que as consultas são realizadas mesmo quando não existem alterações relevantes. Por outro lado, os *webhooks* permitem que a comunicação entre sistemas ocorra apenas quando um evento acontece, tornando o processo mais eficiente, tal como ilustrado na Figura 3 (Steffi, 2025).

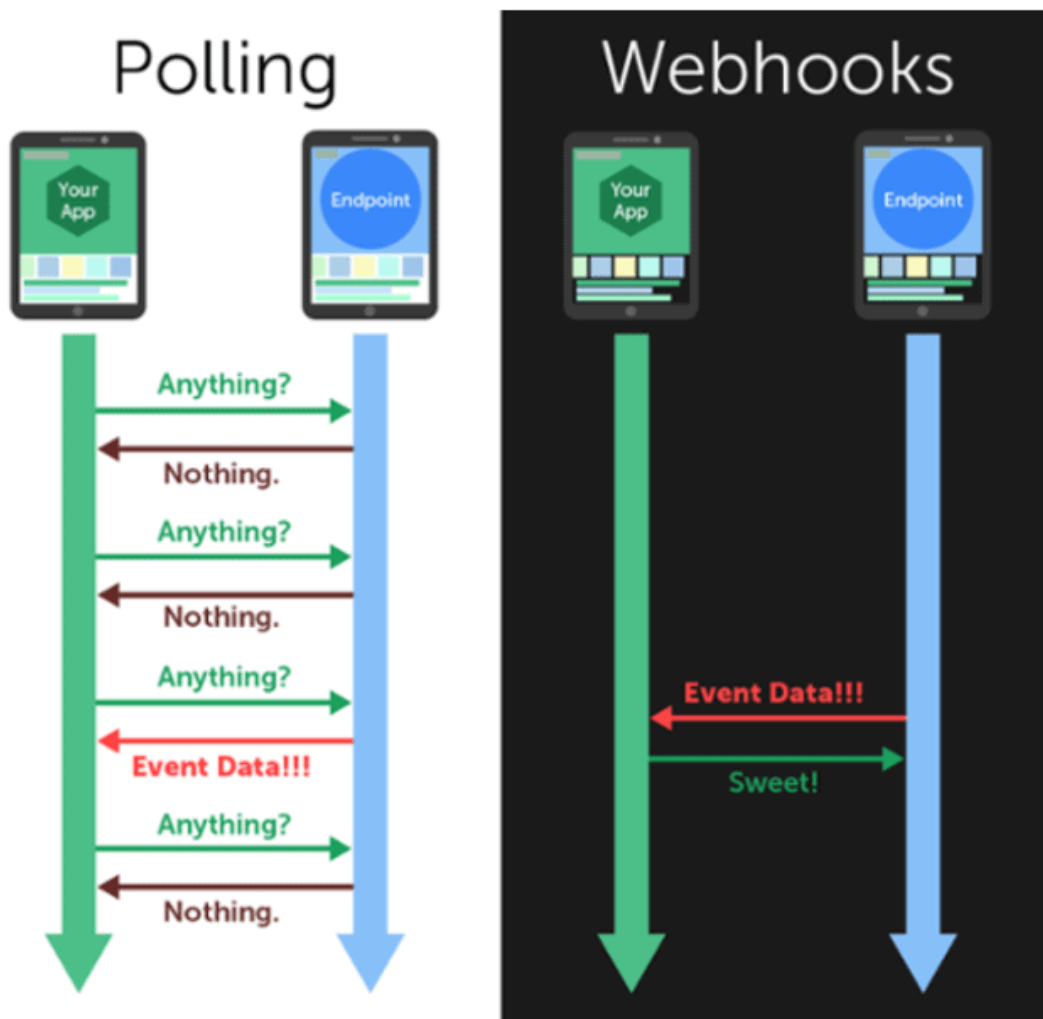


Figura 3 - Polling x Webhooks

2.2.3 Plataformas de automação de workflows

Com o crescimento da integração entre sistemas digitais, surgiram várias ferramentas destinadas à automação de processos e à orquestração de *workflows*, permitindo definir fluxos de trabalho automáticos (IBM Cloud Education Team, 2024).

Um *workflow* automatizado corresponde a uma sequência de passos previamente configurados, geralmente iniciada por um evento, ou *trigger*, que desencadeia um conjunto de ações automáticas. Assim, quando ocorre determinado evento num sistema, o fluxo é executado sem necessidade de intervenção humana.

Este tipo de automação é especialmente relevante em contextos empresariais, onde existe frequentemente a necessidade de integrar diferentes aplicações, reduzir tarefas repetitivas, diminuir erros humanos e aumentar a eficiência dos processos.

Atualmente, existem várias plataformas para criação e gestão de *workflows* automatizados, como Zapier, Make e n8n, que permitem ligar aplicações através de APIs ou *webhooks*.

No âmbito deste projeto, foi adotada a plataforma n8n, por indicação da empresa, em modo *self-hosted*, garantindo que os dados permanecem em servidores próprios. A Tabela 1 apresenta alguns exemplos de eventos que podem ocorrer num sistema CRM e respetivas automações possíveis.

Tabela 1 - Exemplos de automação em sistemas CRM

Evento no sistema	Ação automática	Benefício
Novo cliente registado	Envio de mensagem de boas-vindas	Melhor experiência inicial do cliente
Alteração de dados do cliente	Atualização automática de registos noutras plataformas	Dados sempre consistentes
Cliente sem compras durante X tempo	Geração de alerta ou campanha de reativação	Recuperação de clientes inativos
Novo contacto criado	Notificação da equipa comercial	Resposta comercial mais rápida

2.3 Contexto tecnológico

Esta secção caracteriza o ecossistema tecnológico da XD Software, descrevendo a infraestrutura existente e os padrões de comunicação adotados, que constituem a base técnica sobre a qual a solução foi desenvolvida.

2.3.1 Infraestrutura tecnológica utilizada na XD Software

O desenvolvimento do presente projeto insere-se no ecossistema tecnológico utilizado pela XD Software, que é composto por diferentes aplicações e serviços responsáveis pela gestão de processos empresariais e pela integração entre sistemas.

No centro deste ecossistema encontra-se o sistema ERP Soba, utilizado para a gestão de diversas operações empresariais, tais como registo de clientes, gestão de documentos comerciais, controlo financeiro e outras funcionalidades relacionadas com a atividade das organizações. Este sistema funciona como o principal repositório de dados da plataforma.

Para permitir a comunicação entre o ERP e outras aplicações, a XD Software disponibiliza uma API, que permite acessar e manipular diferentes recursos do sistema de forma programática. Através desta interface é possível consultar, criar ou atualizar dados, permitindo a integração do sistema de gestão com outras ferramentas ou serviços externos.

Para além disso, a empresa dispõe também de soluções destinadas à automação de processos, que permitem reagir automaticamente a determinados acontecimentos registados no sistema. Sempre que ocorre um evento relevante, como a criação ou alteração de determinados dados, é possível desencadear processos automáticos que executam diferentes ações, contribuindo para a integração entre aplicações e para a automatização de tarefas associadas à gestão empresarial.

Esta infraestrutura tecnológica permite assim suportar um modelo de funcionamento baseado na integração entre sistemas e na automação de processos, possibilitando a criação de soluções que respondem automaticamente a diferentes acontecimentos registados no sistema de gestão.

Para além dos padrões referidos, a solução recorre ao Redis como sistema de mensagens e armazenamento temporário de estado, utilizado para gerir a publicação de eventos entre o ERP e o motor de automação.

2.3.2 Tecnologias e padrões de comunicação

A comunicação entre os diferentes componentes do ecossistema tecnológico da XD Software baseia-se em padrões amplamente utilizados no desenvolvimento de aplicações web. Estes padrões permitem garantir que diferentes sistemas consigam comunicar entre si de forma consistente e independente da tecnologia utilizada.

Um dos principais modelos utilizados é o REST (Representational State Transfer), que estabelece um conjunto de princípios para a criação de serviços web baseados no protocolo HTTP. Neste modelo, os recursos do sistema são disponibilizados através de endpoints que podem ser acessados por diferentes aplicações, permitindo a execução de operações como consulta, criação ou atualização de dados.

Para a representação e troca de informação entre sistemas é utilizado frequentemente o formato JSON (JavaScript Object Notation). Este formato permite estruturar dados de forma leve e facilmente interpretável por diferentes linguagens de programação, sendo amplamente utilizado em API e integrações entre aplicações.

No que diz respeito à segurança das comunicações, são utilizados mecanismos de autenticação que garantem que apenas sistemas autorizados conseguem aceder aos recursos disponibilizados pela API. Um dos mecanismos utilizados é o JWT (JSON Web Token), que permite validar a identidade das aplicações que realizam pedidos ao sistema, garantindo assim a integridade e a autenticidade da informação transmitida.

A utilização destes padrões tecnológicos facilita a integração entre diferentes aplicações e permite que os sistemas desenvolvidos pela empresa comuniquem entre si de forma eficiente, constituindo a base tecnológica para a implementação de soluções de automação e integração entre sistemas (XD Software, 2024).

3 Análise e desenho da solução

O presente capítulo descreve o processo de análise e especificação da solução desenvolvida no âmbito do projeto XD Fusion, focando-se na área de automação de CRM. Partindo da compreensão do domínio do problema, são identificados e especificados os requisitos do sistema, seguindo-se a descrição dos casos de uso que enquadram as funcionalidades implementadas e, por fim, a apresentação do desenho arquitetural que sustenta a solução.

A análise de requisitos constitui uma etapa fundamental no desenvolvimento de software, dado que permite transformar necessidades de negócio em especificações técnicas claras e verificáveis. Os requisitos de um sistema descrevem o que este deve fazer, as restrições a que está sujeito e as propriedades que deve apresentar.

A adoção desta abordagem estruturada permite garantir que a solução desenvolvida responde, de forma precisa, às necessidades identificadas.

3.1 Domínio do problema

O domínio do problema enquadra-se na gestão automatizada do relacionamento com clientes, integrando o sistema ERP Soba com uma plataforma de orquestração de *workflows* e serviços externos de comunicação. A solução responde a dois tipos de eventos do ciclo de vida do cliente, sendo estes o seu registo e a deteção de inatividade comercial, executando automaticamente um conjunto de ações sem intervenção manual.

O cliente constitui a entidade central do domínio, representando uma pessoa singular ou coletiva registada no sistema ERP. Os atributos com maior relevância para os processos de automação são o endereço de e-mail, o número de telefone, o distrito (*State*), e o indicador de consentimento de comunicações de marketing (*AcceptsMarketing*). O campo *State* é determinante para a resolução do gestor comercial responsável.

A zona de vendas (*SalesZone*) representa uma área geográfica de atuação comercial, configurada para corresponder aos distritos de Portugal Continental e às regiões autónomas. A correspondência entre o distrito do cliente e a zona é realizada por normalização de texto, garantindo robustez face a variações de capitalização ou acentuação. Cada zona possui um gestor comercial por defeito (*defaultSalesmanId*).

O gestor comercial é o utilizador do ERP responsável pelo acompanhamento dos clientes de uma zona geográfica. O seu endereço de e-mail, obtido via API interna a partir do *defaultSalesmanId*, é o canal de envio das notificações automáticas.

O evento de domínio representa uma ocorrência no sistema ERP que inicia um fluxo de automação. São processados dois tipos: *customer.registered*, gerado em tempo real aquando do registo de um novo cliente, e *customer.inactivity.detected*, gerado periodicamente com uma lista de clientes sem atividade comercial recente.

O *webhook* é o mecanismo pelo qual o sistema XD Cloud comunica os eventos à plataforma n8n, através de um pedido HTTP POST com os dados do evento em JSON.

O *workflow* de automação, implementado em n8n, processa o evento recebido e executa as ações correspondentes: notificação ao gestor, envio de mensagens ao cliente e sincronização com o Brevo, conforme os atributos e o tipo de evento.

A plataforma Brevo é o serviço de marketing digital utilizado para manter a base de contactos atualizada, sincronizando os dados dos clientes que consentiram o recebimento de comunicações comerciais.

A Figura 4 apresenta o modelo de domínio que sintetiza as relações entre as entidades descritas.

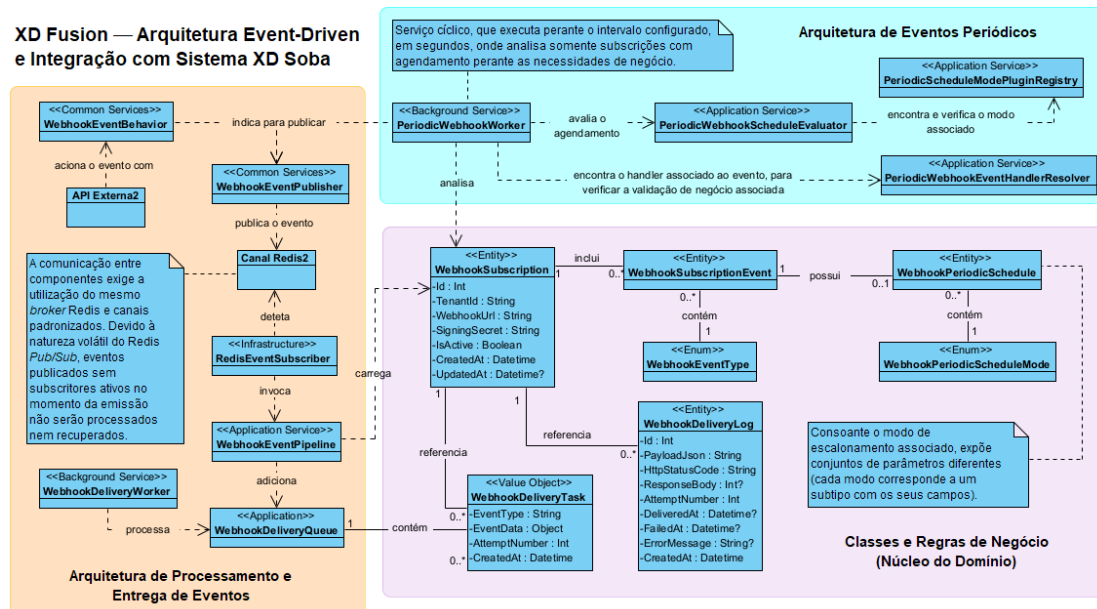


Figura 4 - Domain Model

3.2 Requisitos funcionais e não funcionais

O levantamento de requisitos partiu da análise das necessidades operacionais da XD Software no âmbito da gestão de clientes, tendo como base os dois fluxos de automação identificados no domínio do problema. Para garantir uma especificação abrangente, os requisitos foram organizados segundo o modelo FURPS+ (Grady, 1992). A Tabela 2 descreve as categorias que compõem este modelo (Arham Ehsan, 2025):

Tabela 2 - Categorias do modelo FURPS+

Categoria	Descrição
<i>Functionality</i>	Define o que o sistema deve fazer — as funções e comportamentos esperados.
<i>Usability</i>	Abrange a facilidade de uso, configuração e a qualidade do feedback ao utilizador.
<i>Reliability</i>	Diz respeito à capacidade do sistema em lidar com falhas e garantir a integridade dos dados.
<i>Performance</i>	Cobre os tempos de resposta, a eficiência e o impacto no desempenho dos sistemas envolvidos.
<i>Supportability</i>	Engloba a manutenibilidade, extensibilidade e facilidade de diagnóstico.
+	Inclui restrições de design, segurança, implementação e testabilidade

3.2.1 Requisitos funcionais

Com base nos dois eventos de domínio identificados, foram levantados os requisitos funcionais apresentados na Tabela 3:

Tabela 3 - Requisitos funcionais

ID	User Story	Descrição
RF-01	Notificar gestor sobre novo cliente	Quando um novo cliente é registado no ERP, o sistema deve identificar o gestor comercial responsável pela zona geográfica do cliente e notificá-lo por e-mail com os dados do cliente.
RF-02	Boas-vindas ao novo cliente	O sistema deve contactar o novo cliente após o registo: por e-mail caso este exista, ou por WhatsApp como alternativa caso o cliente não possua e-mail, mas tenha número de telefone.
RF-03	Alertar gestor sobre cliente inativo	Quando detetados clientes sem atividade comercial recente, o sistema deve processar cada cliente individualmente, identificar o gestor responsável e enviar-lhe um alerta por e-mail.
RF-04	Sincronização com Brevo	O sistema deve sincronizar o novo cliente com a plataforma Brevo sempre que o campo <i>AcceptsMarketing</i> estiver ativo e o cliente possuir endereço de e-mail.

3.2.2 Requisitos não funcionais

Os requisitos não funcionais foram levantados com base nas restrições e propriedades de qualidade identificadas para o sistema, organizados segundo as categorias "URPS+" do modelo FURPS+. A Tabela 4 apresenta os requisitos identificados:

Tabela 4 - Requisitos não funcionais

<i>ID</i>	<i>Categoria</i>	<i>Descrição</i>
RNF-01	Usabilidade	Os <i>workflows</i> devem ser configuráveis diretamente na plataforma n8n, sem necessidade de alterações ao código do ERP. O sistema deve tolerar a ausência de dados opcionais (e-mail, telefone) sem falhar, terminando o fluxo de forma controlada.
RNF-02	Fiabilidade	A fiabilidade na entrega dos eventos até ao n8n é garantida pela infraestrutura de <i>webhooks</i> . Os <i>workflows</i> processam cada evento de forma isolada, sem dependências entre execuções.
RNF-03	Desempenho	O processamento dos eventos deve ser assíncrono, garantindo que a execução dos <i>workflows</i> não introduz latência perceptível no fluxo de registo de clientes no ERP.
RNF-04	Suportabilidade	Os <i>workflows</i> devem ser estruturados de forma modular, permitindo a incorporação de novos comportamentos associados a eventos futuros sem necessidade de redesenhar a solução existente.
RNF-05	Segurança (+)	A segurança na autenticação dos eventos é garantida pela infraestrutura de <i>webhooks</i> , que valida a assinatura digital antes de entregar o evento ao <i>endpoint</i> n8n.
RNF-06	Design (+)	A solução deve permanecer desacoplada do ERP, comunicando exclusivamente através de eventos <i>webhook</i> .

3.3 Casos de uso

Com base nos requisitos funcionais definidos na secção anterior, foram identificados quatro casos de uso que descrevem os comportamentos esperados do sistema. O diagrama da Figura 5 apresenta os casos de uso e os atores envolvidos.

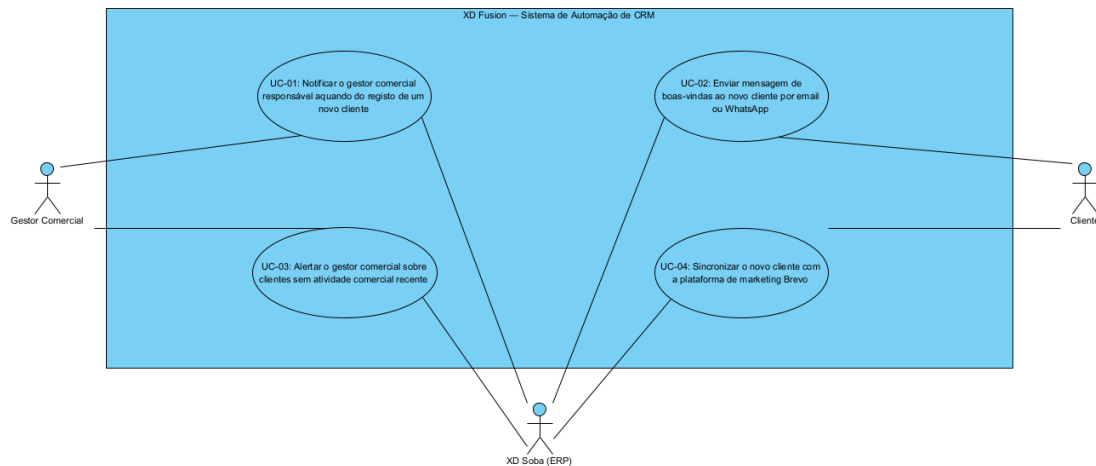


Figura 5 - Use Case Diagram

Os atores identificados são o XD Soba (ERP), responsável por gerar e publicar os eventos que desencadeiam os fluxos de automação, o Gestor Comercial, que recebe as notificações relativas a novos clientes e clientes inativos, e o Cliente, que recebe as mensagens de boas-vindas e cujos dados são sincronizados com o Brevo.

3.3.1 Webhook *customer.registered*

O evento *customer.registered* é publicado pelo ERP XD Soba quando um novo cliente é criado no sistema.

Este evento desencadeia três comportamentos automáticos distintos:

- notificação ao gestor comercial responsável,
- envio de uma mensagem de boas-vindas ao cliente,
- sincronização do contacto com a plataforma Brevo.

A Listagem 1 apresenta o *payload* recebido pelo *webhook*:

Listagem 1 - Payload do webhook *customer.registered*

```

1  {
2    "body": {
3      "event": "customer.registered",
4      "tenantId": "XDPT.58987",
5      "timestamp": "2026-04-30T09:53:06.312204Z",
6      "data": {
7        "KeyId": "9902",
8        "Name": "Cliente teste webhook Setúbal (opt-in)",
9        "BusinessName": "Cliente Teste Setúbal Lda",
10       "Email1": "cliente.setubal.optin@email.com",
11       "Email2": null,
12       "ContactEmail": null,
13       "MobilePhone1": "913456790",
14       "ContactPhone": null,
15       "Phone1": null,
16       "Address": "Avenida Luísa Todi 25",
17       "PostalCode": "2900-450",
18       "City": "Setúbal",
19       "State": "Setúbal",
20       "Country": "PT",
21       "Vat": "PT123456792",
22       "AcceptsMarketing": true,
23       "Inactive": false
24     },
25     "webhookId": "897ff55f573d4c1db74fe2d0d880a70f",
26     "attempt": 1
27   }
28 }

```

3.3.1.1 UC-01: Notificar o gestor comercial sobre registo de um novo cliente

A Figura 6 ilustra o fluxo deste caso de uso.

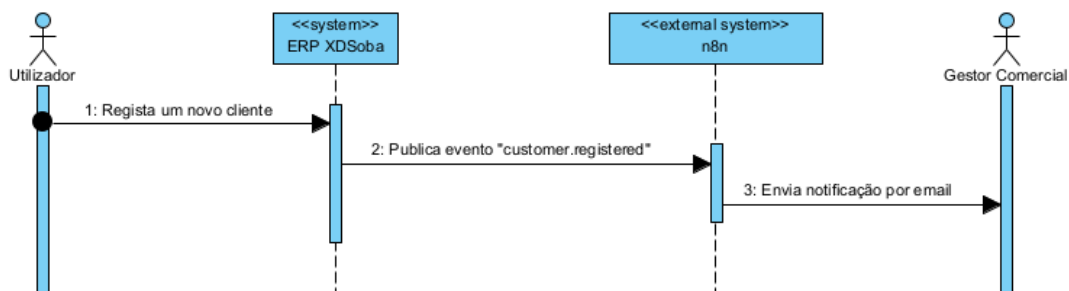


Figura 6 - UC-01: Vista de processos (Nível 1)

Descrição do fluxo

O *workflow* inicia com a extração dos campos relevantes do *payload* recebido, nomeadamente nome, e-mail, telefone, identificador (*KeyId*), cidade, distrito (*State*), país e consentimento de marketing (*AcceptsMarketing*), normalizando-os para uso nos passos seguintes.

De seguida, o *workflow* autentica-se na API interna através de um pedido POST */Auth/combined-login*, obtendo um *token* JWT que é incluído nos cabeçalhos de todos os pedidos subsequentes. Com o *token* obtido, é feito um pedido GET */sales-zone* para recuperar a lista completa de zonas de vendas. A correspondência entre o distrito do cliente e a zona de vendas é determinada por comparação normalizada (remoção de acentos e conversão para minúsculas), garantindo que variações de capitalização ou acentuação nos dados do ERP não causam falhas na resolução.

Identificada a zona correspondente, é efetuado um pedido GET */sales-zone/{zoneId}* para obter o *defaultSalesmanId* associado, seguido de um pedido GET */user/{defaultSalesmanId}* para recuperar os dados completos do gestor. O fluxo verifica então se o gestor possui endereço de e-mail registado:

- caso afirmativo, é enviada uma notificação por e-mail com o nome, identificador, contactos, localização e zona comercial do cliente, recomendando o início do acompanhamento comercial,
- Caso o gestor não possua e-mail, o fluxo termina de forma controlada sem envio.

3.3.1.2 UC-02: Enviar mensagem de boas-vindas a cliente por e-mail ou WhatsApp

A Figura 7 ilustra o fluxo deste caso de uso.

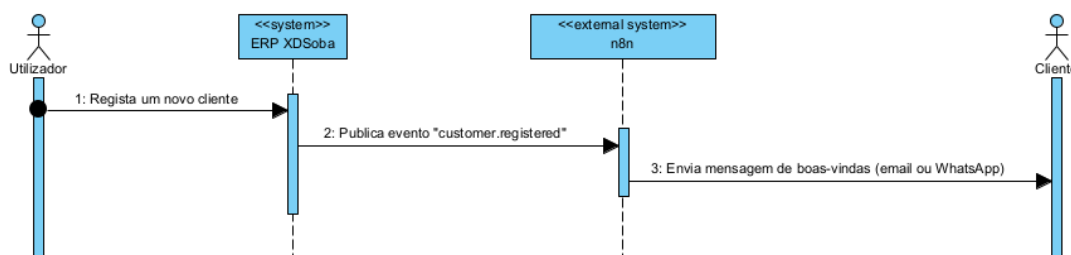


Figura 7 - UC-02: Vista de processos (Nível 1)

Descrição do fluxo

Após a extração dos campos do *payload*, o *workflow* verifica se o cliente possui endereço de e-mail:

- Caso afirmativo, é enviado um e-mail de boas-vindas personalizado com o nome, referência, contactos e localização do cliente, dando as boas-vindas à plataforma XD e apresentando os próximos passos,
- Caso o cliente não possua endereço de e-mail, o *workflow* avalia uma condição alternativa: a existência de número de telefone. Se este estiver disponível, é enviada uma mensagem de boas-vindas via WhatsApp com conteúdo equivalente ao do e-mail. Esta lógica de *fallback* garante que o cliente é sempre contactado pelo canal disponível, sem que a ausência de um meio inviabilize a comunicação. Se nenhum contacto estiver disponível, o fluxo termina de forma controlada sem produzir erros.

3.3.1.3 UC-04: Sincronizar novo cliente com plataforma de marketing

A Figura 8 ilustra o fluxo deste caso de uso.

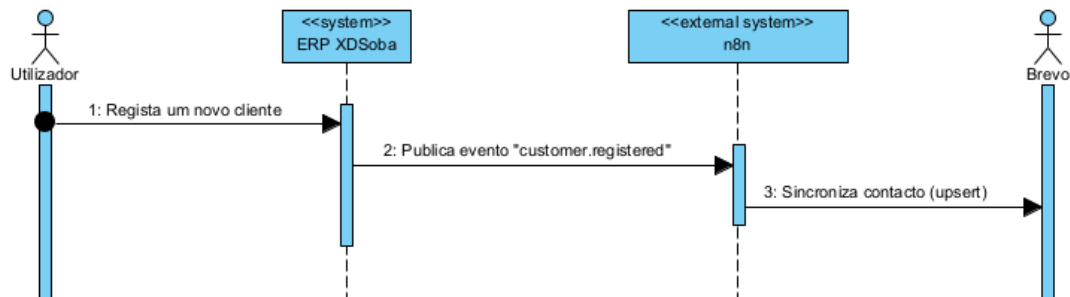


Figura 8 - UC-04: Vista de processos (Nível 1)

Descrição do fluxo

Este caso de uso é executado em continuidade com o UC-02. Após a verificação da existência do e-mail, o *workflow* avalia se o campo *AcceptsMarketing* está ativo.

- Caso afirmativo, é efetuada uma operação de *upsert* na plataforma Brevo através do nó *n8n-nodes-base.sendInBlue*, criando o contacto caso não exista ou atualizando-o

caso já esteja registado. Esta abordagem garante que não existem duplicados na base de contactos de marketing, independentemente do número de vezes que o evento seja processado.

- Caso o cliente não tenha dado consentimento, o fluxo termina sem sincronização, respeitando a preferência do cliente.

3.3.2 Webhook *customer.inactive.detected*

O evento *customer.inactivity.detected* é publicado periodicamente pelo serviço *PeriodicWebhookWorker* do sistema XD Cloud, com base na análise do histórico de compras dos clientes.

O *payload* transporta uma lista de clientes que ultrapassaram o limiar de inatividade calculado individualmente, desencadeando o envio de alertas aos gestores comerciais responsáveis.

A Listagem 2 apresenta o *payload* recebido pelo *webhook*, cuja dimensão é tão elevada que foi necessário cortar de modo a aparecer apenas um dos clientes recebidos:

Listagem 2 – Parte do *payload* do *webhook* *customer.inactivity.detected*

```
1 {
2   "body": {
3     "event": "customer.inactivity.detected",
4     "tenantId": "XDPT.58987",
5     "timestamp": "2026-04-30T10:00:00.000000Z",
6     "data": {
7       "inactiveCustomers": [
8         {
9           "KeyId": "9902",
10          "Name": "Cliente teste webhook Setúbal (opt-in)",
11          "Email1": "cliente.setubal.optin@email.com",
12          "MobilePhone1": "913456790",
13          "City": "Setúbal",
14          "State": "Setúbal",
15          "Country": "PT"
16        }
17      ]
18    },
19    "webhookId": "c20c2802-1fcf-442e-936e-e3a492cb1d67",
20    "attempt": 1
21  }
22 }
```

3.3.2.1 UC-03: Alertar gestor comercial de clientes sem atividade comercial recente

A Figura 9 ilustra o fluxo deste caso de uso.

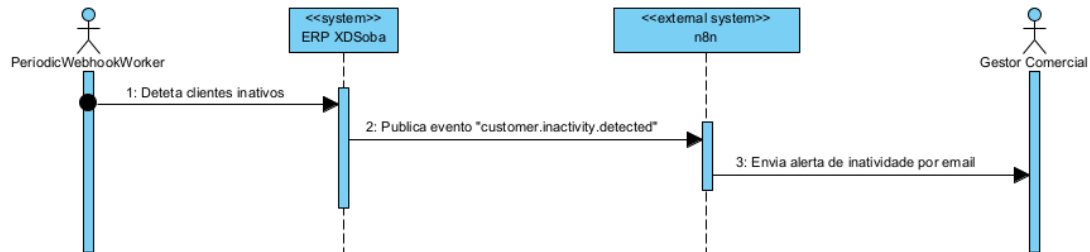


Figura 9 - UC-03: Vista de processos (Nível 1)

Descrição do fluxo

O *workflow* recebe o evento e autentica-se na API interna através de um pedido POST */Auth/combined-login*, obtendo o *token* JWT necessário para os pedidos subsequentes. De seguida, obtém a lista completa de zonas de vendas via GET */sales-zone*.

O *payload* contém um *array inactiveCustomers* com potencialmente múltiplos clientes. O *workflow* itera sobre cada elemento individualmente, processando um cliente de cada vez. Para cada cliente, determina a zona de vendas correspondente ao seu distrito por correspondência normalizada:

- caso a zona não seja encontrada, o cliente em questão é ignorado e o fluxo avança para o seguinte,
- caso seja identificada a zona, é efectuado um pedido GET */user* para obter a lista de utilizadores do sistema, filtrando de seguida o utilizador cujo identificador corresponde ao *defaultSalesmanId* da zona:
 - Caso o gestor identificado possua endereço de email registado, é enviado um alerta por email com os dados do cliente e indicação de necessidade de ação comercial,
 - Caso contrário, o processamento desse cliente termina sem envio e o fluxo prossegue para o cliente seguinte da lista.

3.4 Desenho

O desenho arquitetural da solução é apresentado com recurso ao modelo de vistas 4+1, proposto por Kruchten. Este modelo organiza a descrição da arquitetura em cinco vistas complementares (Kruchten, 1995):

- a **vista lógica**, que descreve a decomposição do sistema em componentes e as suas relações;
- a **vista de implementação**, que apresenta a organização do código em módulos e pacotes;
- a **vista física**, que descreve o *deployment* do sistema na infraestrutura;
- a **vista de processos**, que ilustra os fluxos de execução e as interações dinâmicas entre componentes;
- e a **vista de cenários**, representada pelos casos de uso apresentados na secção anterior, que valida as restantes vistas.

Para a representação das vistas lógica, de implementação e física, é adotada a notação C4, que organiza a descrição em níveis progressivos de detalhe, sendo estes o contexto (N1), containers (N2) e componentes (N3) — permitindo uma leitura gradual da arquitetura (codesee, 2025).

3.4.1 Vista lógica

A vista lógica apresenta o sistema em três níveis de detalhe progressivo.

A Figura 10 apresenta o nível 1, onde o sistema XD Soba é tratado como uma caixa negra, evidenciando apenas as suas fronteiras e a relação com a plataforma de orquestração como entidade externa.

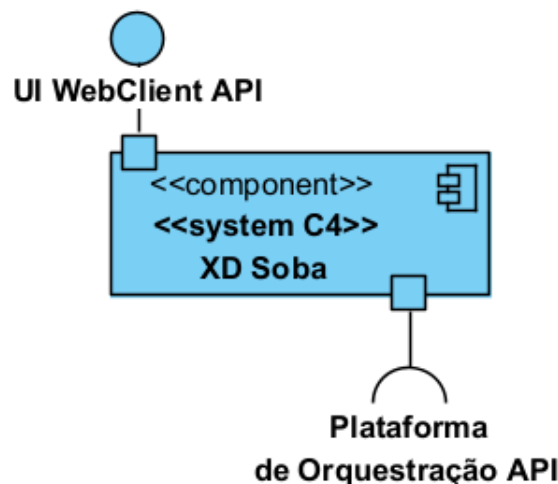


Figura 10 - Vista Lógica (Nível 1)

A Figura 11 expande para o nível 2, onde o sistema é decomposto nos seus containers principais. O *Gateway API* centraliza a comunicação de entrada e saída. O módulo *Webhooks* é o responsável pelo ciclo completo de gestão de eventos — da recepção à entrega. O Canal Redis funciona como barramento de mensagens assíncrono, desacoplando os Publicadores (as restantes aplicações do ecossistema XD) do módulo de processamento. A plataforma n8n surge como sistema externo recetor dos eventos entregues.

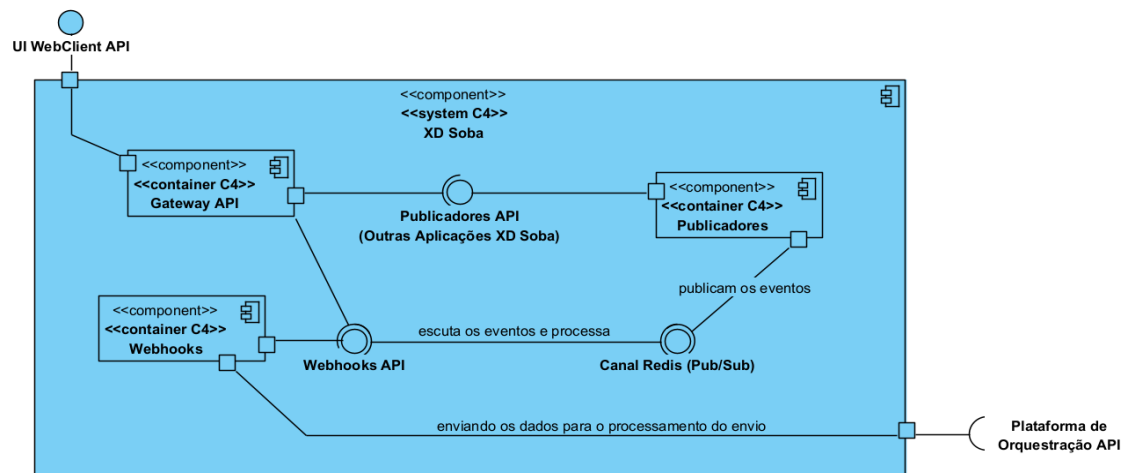


Figura 11 - Vista lógica (Nível 2)

A Figura 12 detalha o nível 3, focando a arquitetura interna do módulo *Webhooks*. A organização segue os princípios da *Clean Architecture*, com quatro camadas concêntricas cujas dependências fluem sempre de fora para dentro:

- **Infrastructure layer:** concentra as implementações técnicas concretas, nomeadamente a ligação ao Redis, ao Azure Table e ao driver da plataforma de orquestração — protegendo o restante sistema de alterações tecnológicas.
- **Interface Adapters Layer:** traduz os dados entre o formato externo e o modelo interno, alojando os *controllers* HTTP, os *background workers* e os repositórios.
- **Application Layer:** orquestra a lógica de negócio através de pipelines de processamento e *handlers* de eventos, sem conhecimento das tecnologias de infraestrutura.
- **Domain Layer:** encapsula as entidades, agregados e contratos do sistema, sendo completamente autónomo face a qualquer dependência externa.

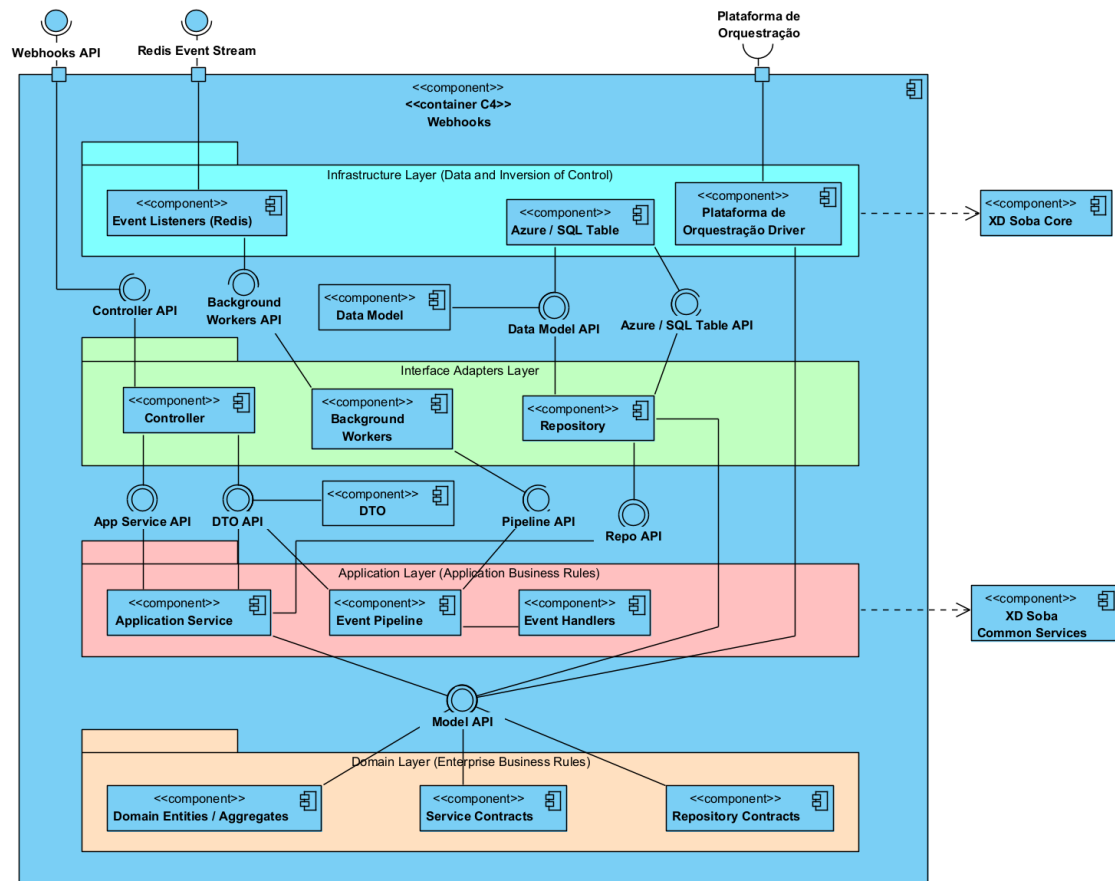


Figura 12 - Vista Lógica (Nível 3)

3.4.2 Vista de implementação

A vista de implementação descreve a organização física do código no ambiente de desenvolvimento.

A Figura 13 apresenta o nível 2, mostrando a decomposição da solução em subsistemas. O módulo *Webhooks* está isolado num projeto independente, sem referências diretas aos restantes módulos do ecossistema, o que permite a sua compilação e evolução autónoma.

A Figura 14 aprofunda para o nível 3, detalhando a organização interna do módulo. Cada camada da *Clean Architecture* corresponde a um projeto separado, o que torna as regras de dependência verificáveis em tempo de compilação. O projeto de domínio não referencia nenhum outro. A camada de aplicação referencia apenas o domínio. Os adaptadores de interface referenciam a aplicação. A infraestrutura referencia todas as camadas, incluindo um módulo de inversão de controlo (IoC) com visibilidade transversal, que resolve as dependências em tempo de execução.

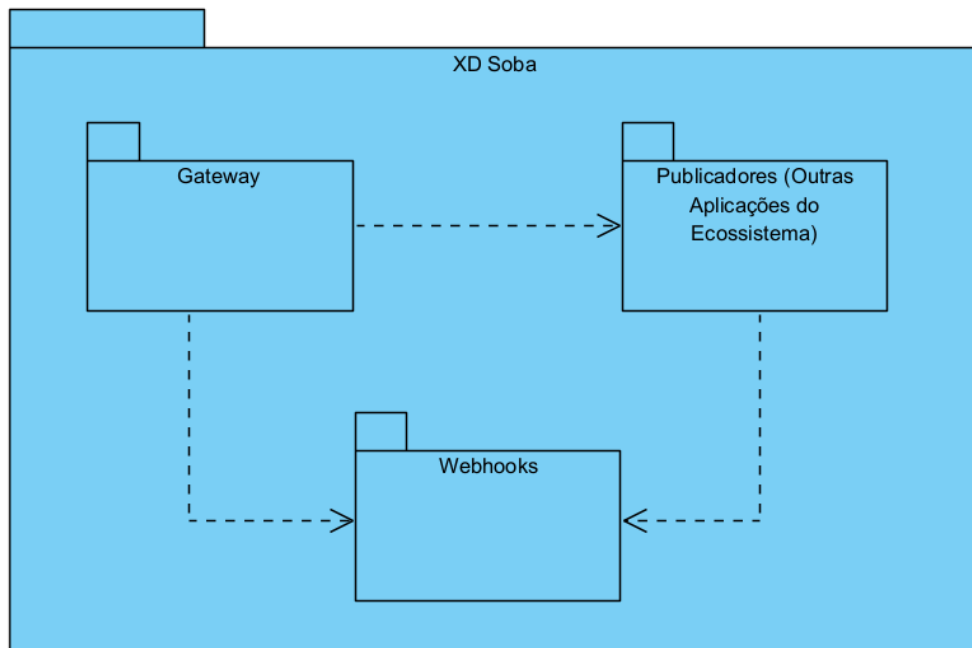


Figura 13 - Vista de Implementação (Nível 2)

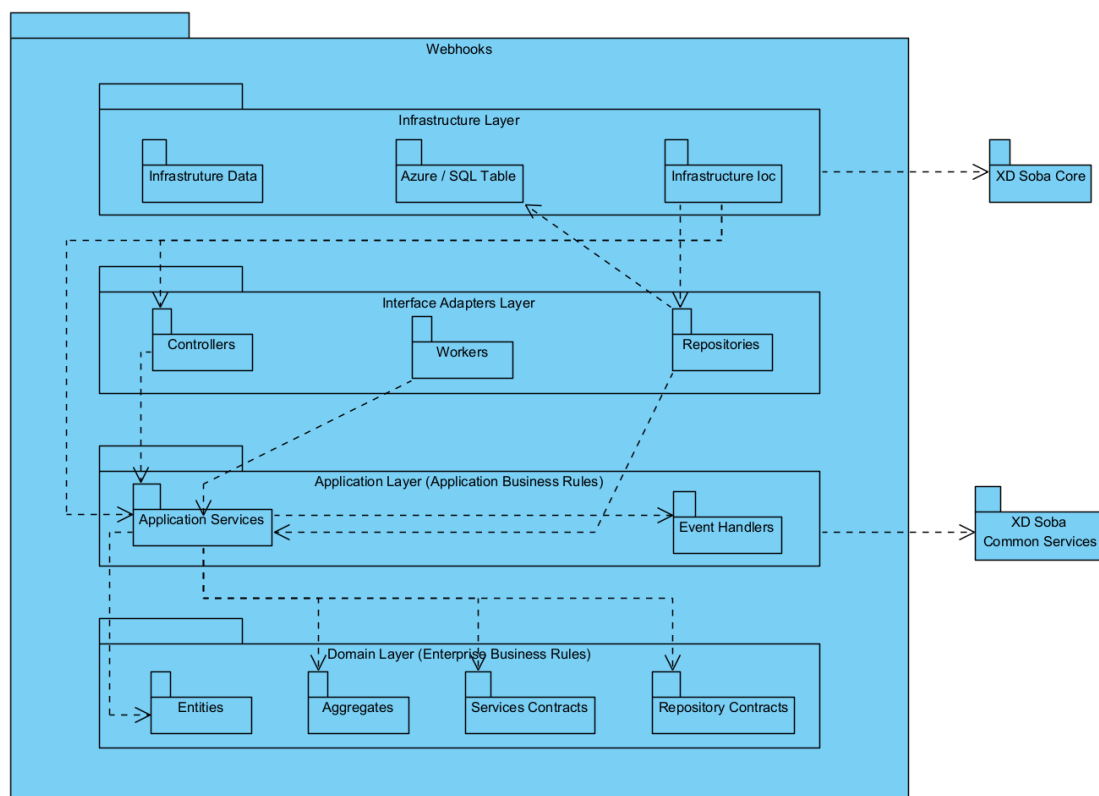


Figura 14 - Vista de Implementação (Nível 3)

3.4.3 Vista física

A Figura 15 descreve a topologia de *deployment* do sistema. Toda a solução corre em ambiente de contentores Docker, num servidor *cloud* ou máquina local.

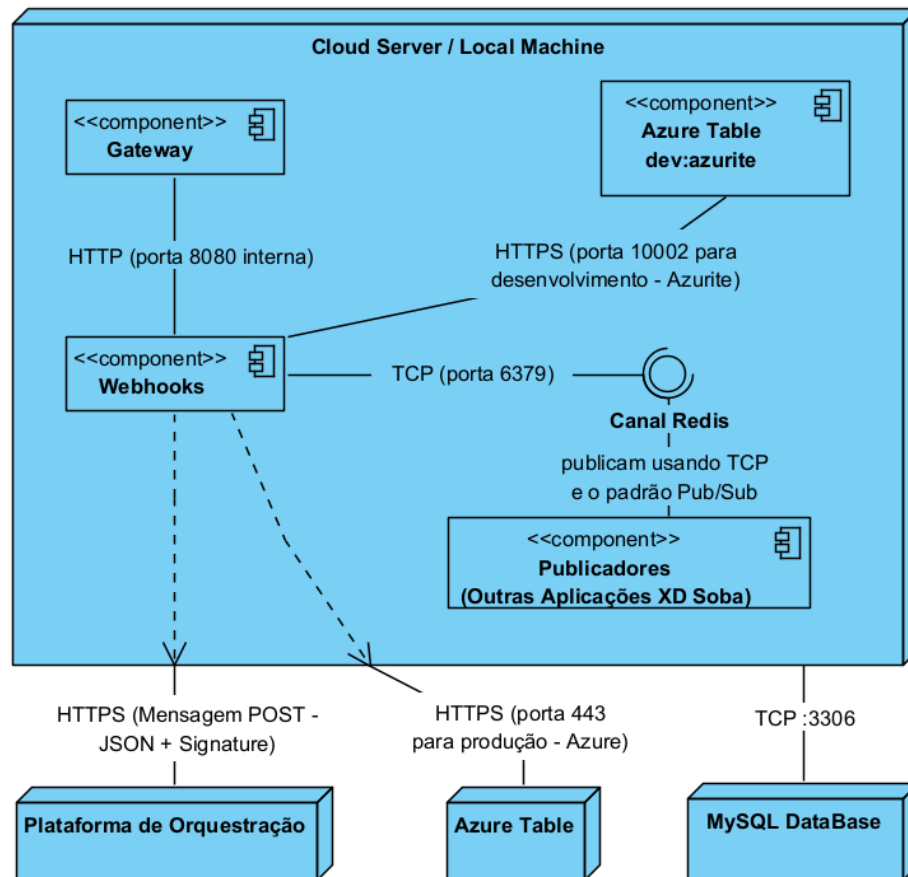


Figura 15 - Vista Física do Sistema

O *Gateway* recebe o tráfego externo e comunica com o módulo *Webhooks* via HTTP interno na porta 8080. Os eventos são propagados internamente através do Canal Redis via TCP na porta 6379. O armazenamento de subscrições é feito no Azure Table Storage via HTTPS em produção, ou no emulador Azurite (porta 10002) em desenvolvimento. A base de dados transacional MySQL é acedida via TCP na porta 3306. A entrega de eventos à plataforma n8n é feita via HTTPS POST com *payload* JSON e assinatura digital, garantindo autenticidade e integridade na comunicação.

3.4.4 Vista de processos

A presente secção descreve o comportamento dinâmico do sistema em tempo de execução, detalhando como os componentes internos do módulo de *Webhooks* colaboram para processar cada evento e entregar o *payload* ao n8n.

A solução assenta em dois fluxos de execução distintos: um fluxo reativo, desencadeado por alterações na base de dados, e um fluxo periódico, gerido por um serviço em segundo plano.

UC-01, UC-02 E UC-04 — Fluxo reativo (*customer.registered*)

O fluxo tem início quando o EF Core persiste um novo cliente na base de dados. O *WebhookEventInterceptor*, implementado como um *SaveChangesInterceptor*, intercepta a operação no evento *SavingChangesAsync*, identifica a entidade como um cliente e regista o evento pendente. Após a conclusão bem-sucedida do *SavedChangesAsync*, o *interceptor* acrescenta ao *payload* o campo *AcceptsMarketing*, cujo valor reflete o consentimento de marketing dado pelo cliente aquando do registo (*true* quando o *EntityConsentmentStatus* é igual ou superior a 1, e *false* caso contrário) e invoca o *WebhookEventPublisher*, que serializa o evento e o publica no canal Redis do *tenant*.

O *RedisEventSubscriber*, um serviço em segundo plano com subscrição ativa ao padrão de canais Redis, deteta a mensagem e delega o processamento ao *WebhookEventPipeline*. Este desserializa o evento, consulta o *IWebhookSubscriptionRepository* para identificar as subscrições ativas para o tipo *customer.registered* do *tenant* em causa, e coloca uma tarefa de entrega na fila do *IWebhookDeliverySink* por cada subscrição encontrada.

Por fim, o *WebhookDeliveryWorker* processa a fila e envia o *payload* via HTTP POST para o *endpoint* configurado no n8n, que executa as automações de notificação ao gestor (US1), boas-vindas ao cliente (US2) e sincronização com o Brevo (US3).

Neste fluxo, a resolução do gestor responsável é efetuada pelo próprio *workflow* do n8n, que obtém a lista de zonas de venda via API e determina a zona correspondente ao distrito do cliente através da mesma lógica de conforme a Figura 16.

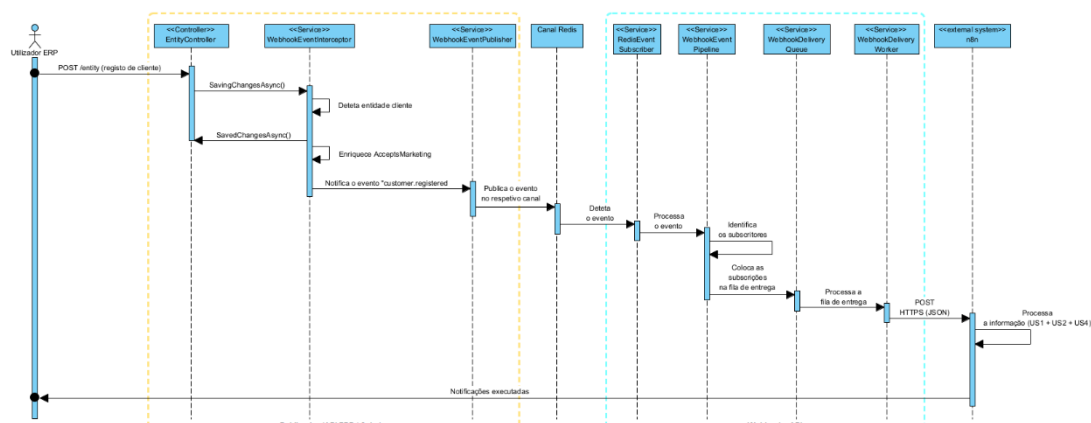


Figura 16 - UC-01-02-04: Vista de Processos (Nível 3)

UC-03 — Fluxo periódico (*customer.inactivity.detected*)

Este fluxo é coordenado pelo *PeriodicWebhookWorker*, um serviço em segundo plano que executa ciclicamente.

No início de cada ciclo, é adquirido um bloqueio exclusivo através do componente *RedisPeriodicWebhookDispatchVerifier*. Este mecanismo garante que apenas uma instância processa o ciclo de verificação, mesmo em ambientes multi-instância.

De seguida, são carregadas as subscrições periódicas *ativas*. Para cada subscrição devida, o *IPeriodicWebhookEventHandlerResolver* identifica o handler responsável, neste caso o *CustomerInactivityDetectedPeriodicWebhookHandler*. Este *handler* delega a deteção de clientes inativos na consulta *CustomerInactiveDetectionQuery*.

A consulta acede à base de dados do respetivo *tenant* e aplica um algoritmo multicritério configurável, baseado no tempo sem compras, no histórico de compras do cliente e no caso particular de clientes sem compras desde o registo. A avaliação individual é depois delegada ao *CustomerInactivityEvaluator*.

Para cada cliente inativo, o gestor responsável é resolvido por correspondência normalizada entre o distrito do cliente e as zonas de venda configuradas. O *RedisCustomerInactivityNotificationLedger* filtra clientes já notificados, e o *payload* é publicado no Redis percorrendo o mesmo pipeline de entrega descrito anteriormente. O registo no *ledger* ocorre apenas após o n8n confirmar o envio via POST */api/v1/periodic/inactivity-ack*.

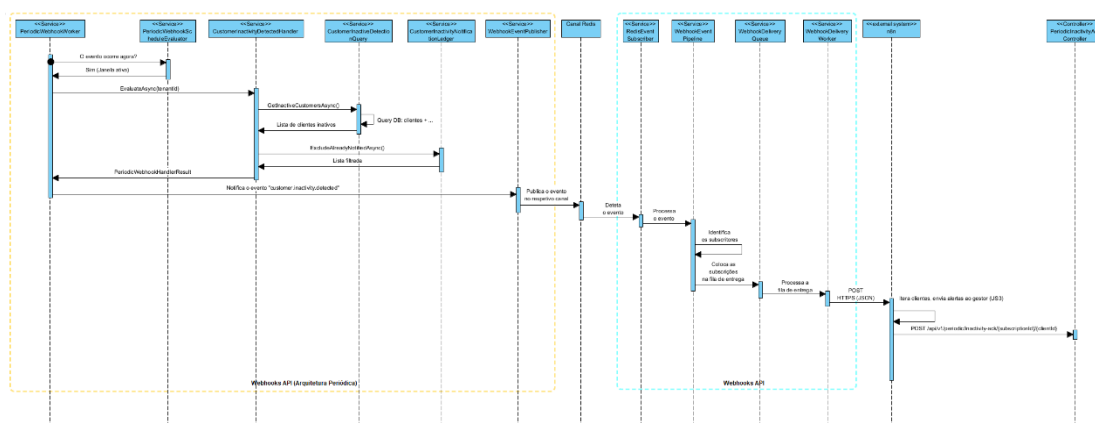


Figura 17 - UC-03: Vista de Processos (Nível 3)

4 Implementação da Solução

O presente capítulo descreve a implementação da solução proposta no capítulo anterior. São apresentados os detalhes de implementação do módulo de Webhooks — com enfoque nos mecanismos de detecção, publicação e entrega de eventos — e dos workflows de automação desenvolvidos em n8n para responder aos quatro casos de uso de CRM definidos. O capítulo inclui ainda a descrição da estratégia de verificação adotada e uma avaliação da solução face aos requisitos definidos na secção 3.2.

4.1 Descrição da implementação

A implementação da solução dividiu-se em duas frentes complementares e desacopladas: o módulo de *Webhooks* do *backend* .NET e os *workflows* de automação na plataforma n8n. O *backend* é responsável por detetar os eventos relevantes no sistema ERP, publicá-los de forma assíncrona no canal Redis e garantir a sua entrega HTTP ao *endpoint* configurado, o *webhook* do n8n.

O n8n, por sua vez, recebe os *payloads* e executa as ações de negócio correspondentes, consultando a API interna do ERP sempre que necessário para obter dados adicionais.

Esta separação garante que a infraestrutura de eventos é agnóstica à lógica de negócio: adicionar, modificar ou remover um *workflow* no n8n não implica qualquer alteração ao *backend*, e vice-versa.

4.1.1 Tecnologias e metodologias utilizadas

O módulo de *Webhooks* foi implementado em .NET 8 com ASP.NET Core, seguindo os princípios da *Clean Architecture* com quatro camadas independentes:

- domínio,
- aplicação,
- adaptadores de interface,
- infraestrutura.

A organização do código em projetos separados por camada torna as regras de dependência verificáveis em tempo de compilação, nenhum projeto de camada inferior referencia camadas superiores.

A interceção de eventos de persistência recorre ao mecanismo EF Core *SaveChangesInterceptor*, que permite reagir a alterações na base de dados sem acoplar a deteção de eventos à lógica de negócio dos comandos.

A comunicação assíncrona entre produtores e consumidores de eventos é suportada pelo Redis Pub/Sub, acedido através da biblioteca *StackExchange.Redis*.

A orquestração interna de comandos segue o padrão CQRS com a biblioteca MediatR, complementada pelo *FluentValidation* para validação de entradas. O armazenamento de subscrições em produção recorre ao Azure Table Storage; em desenvolvimento local, utiliza-se o emulador Azurite.

A autenticação nas chamadas à API interna é realizada com *tokens* JWT, obtidos por um pedido de login antes de cada fluxo n8n. Para os workflows de automação, utilizou-se a plataforma n8n em modo *self-hosted*.

A estratégia de verificação baseia-se no ecossistema NUnit, FluentAssertions e Moq para testes unitários. O controlo de versões foi gerido com Git, num repositório privado fornecido pela empresa. O desenvolvimento seguiu uma organização iterativa em fases, com definição progressiva de objetivos e revisões periódicas com o supervisor, alinhada com os princípios ágeis adotados pela equipa.

4.1.2 Implementação da arquitetura orientada a eventos

O ponto de partida de qualquer entrega de *webhook* é a subscrição (*WebhookSubscription*). Cada subscrição regista, para um dado *tenant*, o URL de destino, um segredo de assinatura utilizado para autenticar os pedidos entregues, e o conjunto de tipos de evento que devem ser encaminhados para esse URL.

As subscrições são persistidas no Azure Table Storage (emulado localmente com Azurite) e carregadas pelo *IWebhookSubscriptionRepository* em cada ciclo de processamento. Esta separação entre a configuração das subscrições e a lógica de deteção e entrega de eventos permite que novos destinos ou novos tipos de evento sejam adicionados sem qualquer alteração ao código.

A entrada unificada no sistema de publicação é o *WebhookEventPublisher*, que constrói o canal Redis de destino seguindo a convenção *{channelPrefix}:{tenantId}*, onde o prefixo é configurável (por defeito *webhook:events*), serializa o *payload* em JSON e publica-o.

Este componente é injetado em dois pontos distintos do ecossistema, materializando dois fluxos de execução independentes:

- o **fluxo reativo**, desencadeado por alterações na base de dados do ERP,
- e o **fluxo periódico**, gerido por um serviço em segundo plano com agendamento configurável.

Ambos partilham a mesma infraestrutura de publicação, subscrição e entrega HTTP, tal como ilustrado na Figura 18.

4.1.2.1 Fluxo reativo

O fluxo reativo tem origem no *WebhookEventInterceptor*, implementado como um EF Core *SaveChangesInterceptor* registado no *DbContext*. O componente interceta o ciclo de persistência em duas fases: na fase *SavingChangesAsync*, o método *CapturePendingEvents* inspeciona o *ChangeTracker* e regista internamente os eventos pendentes para as entidades nos estados *Added*, *Modified* ou *Deleted* que estejam mapeadas para *webhooks*.

Para entidades do tipo *Customer*, são gerados eventos específicos do domínio CRM (*customer.registered*, *customer.updated* e *customer.deleted*) em adição ao evento genérico *entity.**.

No caso de criação, o *payload* do *customer.registered* é enriquecido com o campo *AcceptsMarketing*, derivado do *EntityConsentmentStatus* do cliente:

- **true** quando o valor é igual ou superior a 1 (consentimento explícito de marketing),
- **false** caso contrário.

Na fase *SavedChangesAsync*, após a conclusão bem-sucedida da escrita, os eventos são extraídos e publicados numa *Task.Run* independente, garantindo que o pedido HTTP do ERP retorna sem aguardar a publicação no Redis e cumprindo assim o RNF-03.

No lado da subscrição, o *RedisEventSubscriber*, um *BackgroundService* com subscrição permanente sobre o padrão *{channelPrefix}:**, deteta as mensagens publicadas e, para cada uma, cria um âmbito de injeção de dependências isolado via *IServiceScopeFactory.CreateScope*, prevenindo conflitos de estado no processamento paralelo. O processamento é delegado ao *WebhookEventPipeline*, que consulta o *IWebhookSubscriptionRepository* para identificar as subscrições ativas e coloca uma

WebhookDeliveryTask na fila por cada uma encontrada, conforme ilustrado na Listagem 3 e na Figura 18.

Listagem 3 - RedisEventSubscriber: isolamento de contexto por mensagem via CreateScope

```
private async Task ProcessEventAsync(RedisChannel channel, RedisValue message, CancellationToken cancellationToken)
{
    var channelName = channel.ToString();
    var rawMessage = message.ToString() ?? string.Empty;

    using var scope = _serviceScopeFactory.CreateScope();
    var pipeline = scope.ServiceProvider.GetRequiredService<IWebhookEventPipeline>();

    await pipeline.ProcessAsync(channelName, rawMessage, cancellationToken);
}
```

A entrega é gerida pelo *WebhookDeliveryWorker*, que controla a concorrência das entregas HTTP com um *SemaphoreSlim* (por defeito 5 entregas simultâneas). Em caso de falha, aplica uma política de reenvio com atrasos progressivos ([0, 30, 300] segundos, até 3 tentativas), regista cada tentativa no *IWebhookDeliveryLogRepository* e mantém apenas os últimos 50 registos por subscrição.

A Figura 18 apresenta uma visão consolidada do pipeline reativo implementado, desde a deteção da alteração no ERP até à entrega HTTP do evento ao *endpoint* configurado no n8n. Esta representação permite compreender a sequência de componentes envolvidos na publicação, processamento e entrega dos eventos, evitando a apresentação fragmentada de excertos de código.

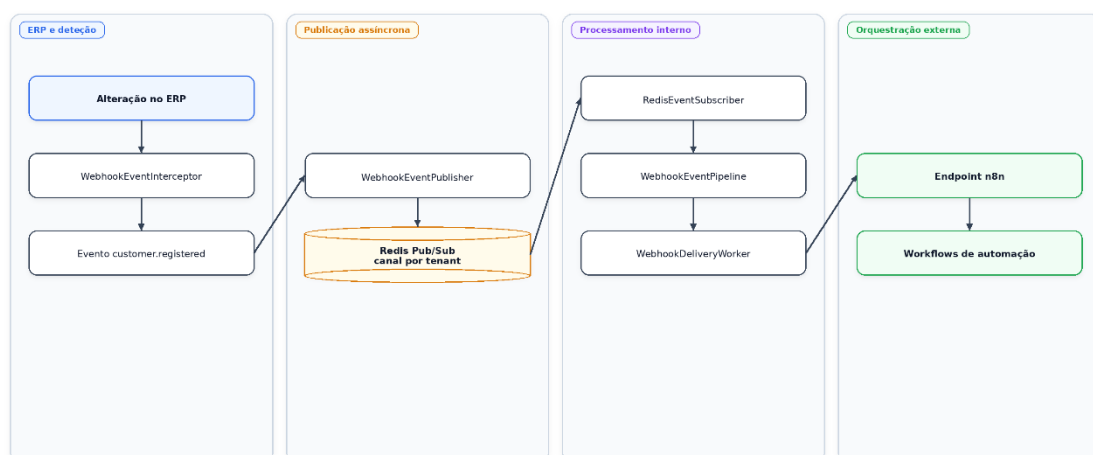


Figura 18 - Pipeline de publicação, processamento e entrega de eventos reativos

4.1.2.2 Fluxo periódico

O fluxo periódico é coordenado pelo *PeriodicWebhookWorker*, um *BackgroundService* que executa ciclicamente. No início de cada ciclo, tenta adquirir um bloqueio exclusivo no Redis via *RedisPeriodicWebhookDispatchVerifier*, se outra instância já o detém, o ciclo é abortado, prevenindo execuções duplicadas em ambientes com múltiplas réplicas, conforme ilustrado na Figura 19.

O sistema suporta três modos de agendamento implementados como plugins independentes (*IPeriodicScheduleModePlugin*): diário, semanal e por intervalo. No âmbito deste projeto foi utilizado o modo semanal, configurado com dia da semana (*DayOfWeek*), hora (HH:mm) e fuso horário (*TimeZoneId*). A avaliação de cada subscrição é delegada ao método *IsDue*, que verifica se o schedule corresponde ao modo semanal configurado, tal como mostra a Listagem 4.

Listagem 4 - WeeklyPeriodicScheduleModePlugin: avaliação da subscrição como devida pelo método IsDue

```
public bool IsDue(PeriodicWebhookEventDefinition definition, DateTimeOffset utcNow) =>
    definition.Schedule is WeeklyWebhookPeriodicSchedule w && IsWeeklyDue(w, utcNow);
```

Para evitar execuções repetidas na mesma semana, é reservado no Redis um slot identificado pela data do dia agendado na semana corrente, e se o slot já existir, o ciclo é ignorado para aquela subscrição.

A Figura 19 sintetiza o funcionamento do fluxo periódico responsável pela deteção de clientes inativos. O processo controla a execução através de um bloqueio no Redis, valida o agendamento configurado, evita execuções repetidas no mesmo *slot* temporal e apenas publica o evento *customer.inactivity.detected* após a avaliação das regras de inatividade comercial.

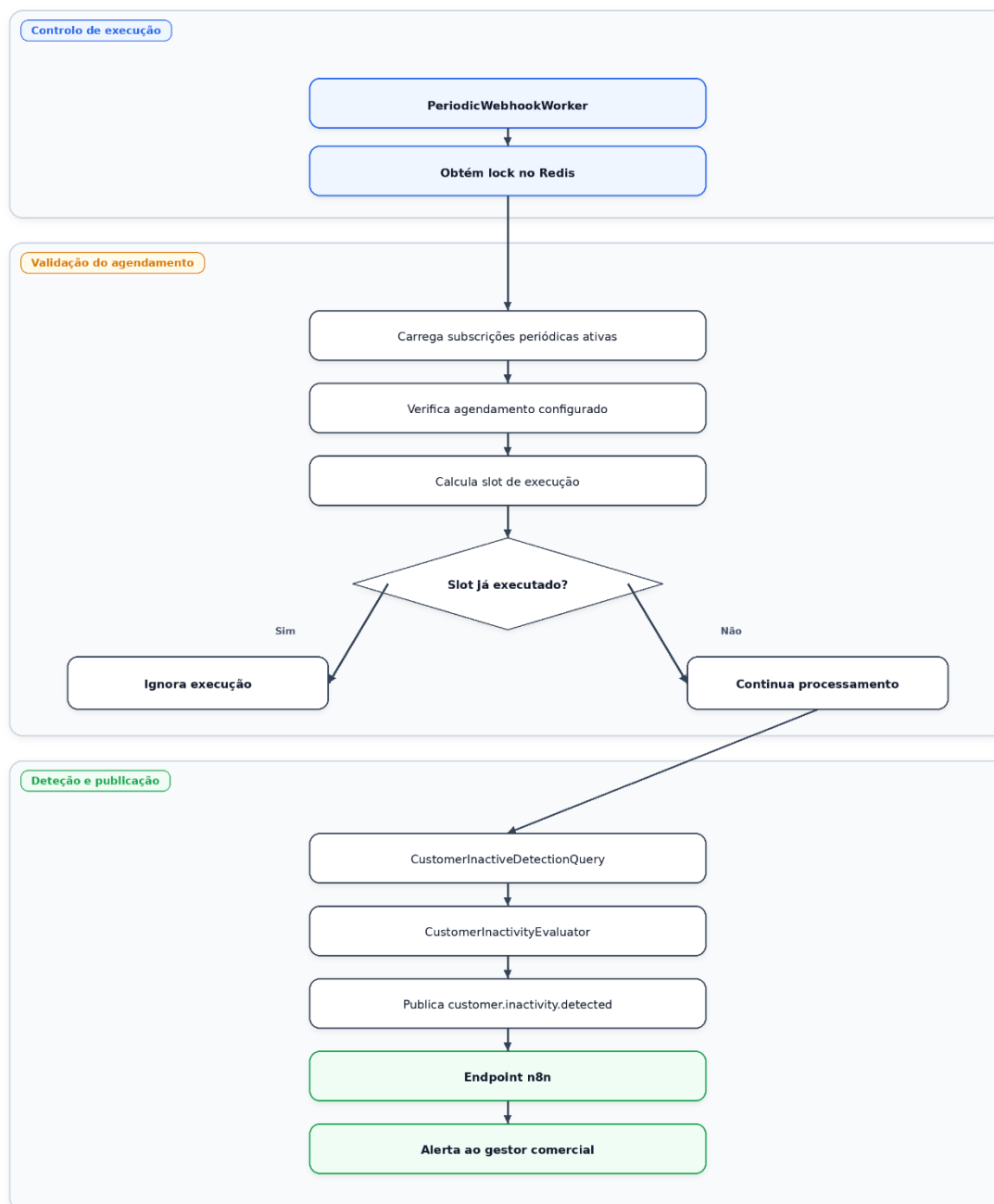


Figura 19 - Fluxo periódico de deteção e publicação de clientes inativos

Para as subscrições devidas, o *IPeriodicWebhookEventHandlerResolver* instancia o *CustomerInactivityDetectedPeriodicWebhookHandler*, que delega a deteção ao *CustomerInactiveDetectionQuery*. A query carrega todos os clientes ativos do *tenant* e o seu histórico de compras dos últimos 15 anos.

A inatividade de cada cliente é avaliada pelo *CustomerInactivityEvaluator* segundo três ramos:

- clientes sem compras (limiar baseado em dias desde o registo),
- clientes com histórico insuficiente (limiar fixo *MinimumInactiveDays*),
- e clientes com histórico suficiente (limiar dinâmico baseado na mediana dos intervalos entre compras multiplicada por *AverageGapMultiplier*).

Este último ramo adapta o limiar de inatividade ao padrão comercial de cada cliente, reduzindo a probabilidade de falsos positivos e falsos negativos. A Tabela 5 sintetiza os três ramos de avaliação implementados pelo *CustomerInactivityEvaluator*, apresentando a lógica de decisão de forma mais clara e direta do que a exposição detalhada do código.

Tabela 5 - Regras de avaliação de inatividade comercial

Situação avaliada	Regra aplicada	Resultado esperado
Cliente sem histórico de compras	Avalia o tempo decorrido desde a criação do cliente	Pode gerar alerta se ultrapassar o limite configurado
Cliente com histórico insuficiente	Aplica uma regra conservadora definida na subscrição	Evita decisões frágeis com base em poucos dados
Cliente com histórico suficiente	Compara o intervalo atual sem compras com o padrão histórico do cliente	Gera alerta se o desvio indicar inatividade relevante

Para cada cliente inativo, o gestor responsável é resolvido por correspondência normalizada entre o distrito do cliente (*State*) e o nome das *SalesZones*, aplicando decomposição Unicode para remover diacríticos e convertendo para minúsculas, lógica ilustrada na Listagem 5.

Listagem 5 - CustomerInactiveDetectionQuery: normalização Unicode do distrito para correspondência com zonas de venda

```
private static string NormalizeDistrict(string? text)
{
    if (string.IsNullOrEmpty(text))
        return string.Empty;

    var decomposed :string = text.Trim().Normalize(NormalizationForm.FormD);
    var sb = new StringBuilder(decomposed.Length);
    foreach (var ch :char in decomposed)
    {
        if (CharUnicodeInfo.GetUnicodeCategory(ch) != UnicodeCategory.NonSpacingMark)
            sb.Append(ch);
    }
    return sb.ToString().Normalize(NormalizationForm.FormC).ToLowerInvariant();
}
```

O *RedisCustomerInactivityNotificationLedger* filtra os clientes já notificados no ciclo atual. O *payload* final é publicado no Redis e percorre o mesmo pipeline do fluxo reativo até ao n8n. O registo no *ledger* ocorre apenas após o n8n confirmar o envio via POST */api/v1/periodic/inactivity-ack*, assim garantindo que falhas no *workflow* não marcam incorretamente o cliente como notificado.

4.1.3 Casos de uso e fluxos de negócio

Nesta secção detalha-se a aplicação prática da arquitetura desenvolvida para satisfazer os requisitos funcionais do projeto. Cada caso de uso é acompanhado por evidências que demonstram o fluxo desde a origem no ERP até à entrega final ao gestor ou ao cliente.

4.1.3.1 UC-01: Notificar o gestor comercial sobre um novo cliente

Deteção e publicação do evento

O processo tem início quando um novo cliente é persistido na base de dados do ERP. O *WebhookEventInterceptor* deteta a entidade *Customer* no estado *Added*, constrói o *payload* com os dados do cliente e enriquece-o com o campo *AcceptsMarketing*, derivado do *EntityConsentmentStatus*. O evento *customer.registered* é publicado de forma assíncrona no

canal Redis do *tenant*, sem bloquear o pedido HTTP do ERP, tal como detalhado na figura 19 do módulo 4.1.1.

Orquestração no n8n

Após a receção do evento, o *workflow* autentica-se na API interna via POST */Auth/combined-login*. De seguida, obtém todas as zonas de vendas via GET */sales-zone* e determina a zona correspondente ao distrito do cliente por comparação normalizada. Identificada a zona, obtém o gestor via GET */user/{defaultSalesmanId}* e envia-lhe uma notificação por e-mail com os dados do cliente. Caso o gestor não possua e-mail, o fluxo termina de forma controlada. As figuras 20 e 21, respetivamente, mostram o fluxo desde a receção pelo *webhook*, e o e-mail recebido.

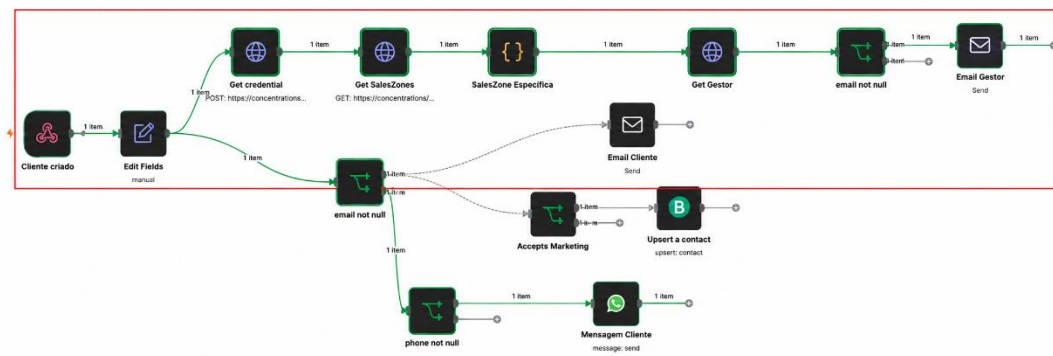


Figura 20 - UC-01: fluxo n8n

Novo Cliente Registrado

Foi identificado um novo registo no sistema comercial

Olá,

foi criado um novo cliente na plataforma XD, recomendando-se o início do acompanhamento comercial com a maior brevidade possível.

DADOS DO CLIENTE

Nome: Cliente teste webhook Setúbal (opt-in)

Identificador: 9902

Email: cliente.setubal.optin@email.com

Telefone: 913456790

Cidade: Setúbal

Distrito: Setúbal

País: PT

SetúbalZona Comercial:

Ação Recomendada:

Proceder ao contacto inicial e acompanhamento comercial deste cliente, garantindo uma integração rápida e uma relação comercial estruturada desde o primeiro momento.

Este alerta foi gerado automaticamente pelo sistema XD.

Tecnologia de automação comercial fornecida por XD Software

Figura 21 - E-mail gestor de novo cliente

Contratempos e soluções

O principal desafio deste fluxo foi a resolução do gestor responsável. Os campos *SalesMan* e *SalesZoneId* presentes na entidade do cliente no ERP não garantem fiabilidade.

A solução passou por ignorar esses campos e resolver sempre o gestor a partir do distrito (*State*) do cliente, normalizando o texto para garantir correspondência robusta independentemente de variações de capitalização ou acentuação nos dados do ERP.

4.1.3.2 UC-02: Enviar mensagem de boas-vindas ao novo cliente

Deteção e publicação do evento

O fluxo partilha o mesmo evento *customer.registered* do UC-01, gerado pelo mesmo mecanismo de interceção. Não existe publicação adicional, o mesmo *payload* é entregue ao n8n, que executa este *workflow* em paralelo com o UC-01 e UC-04.

Orquestração no n8n

O *workflow* verifica se o cliente possui endereço de e-mail:

- em caso **afirmativo**, envia uma mensagem de boas-vindas personalizada (demonstrado na figura 22 e na figura 23),
- na **ausência** de e-mail, avalia a existência de número de telefone e, se disponível, envia a mensagem via WhatsApp (demonstrado na figura 24 e na figura 25).
- Se nenhum contacto estiver disponível, o fluxo termina sem erros (demonstrado na figura 26).

E-mail

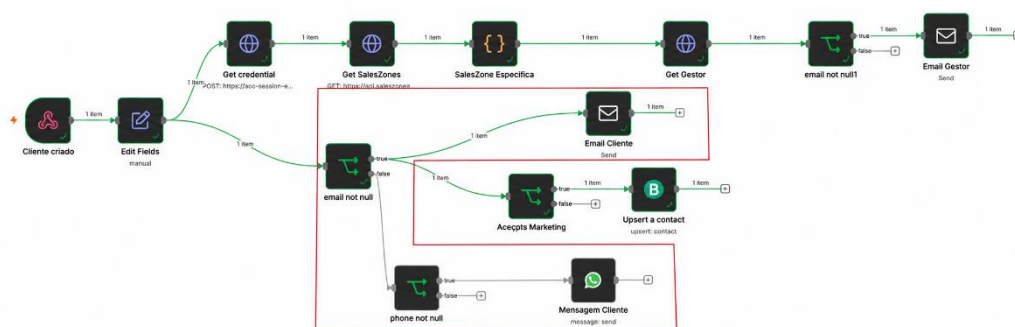


Figura 22 - UC-02: fluxo n8n e-mail

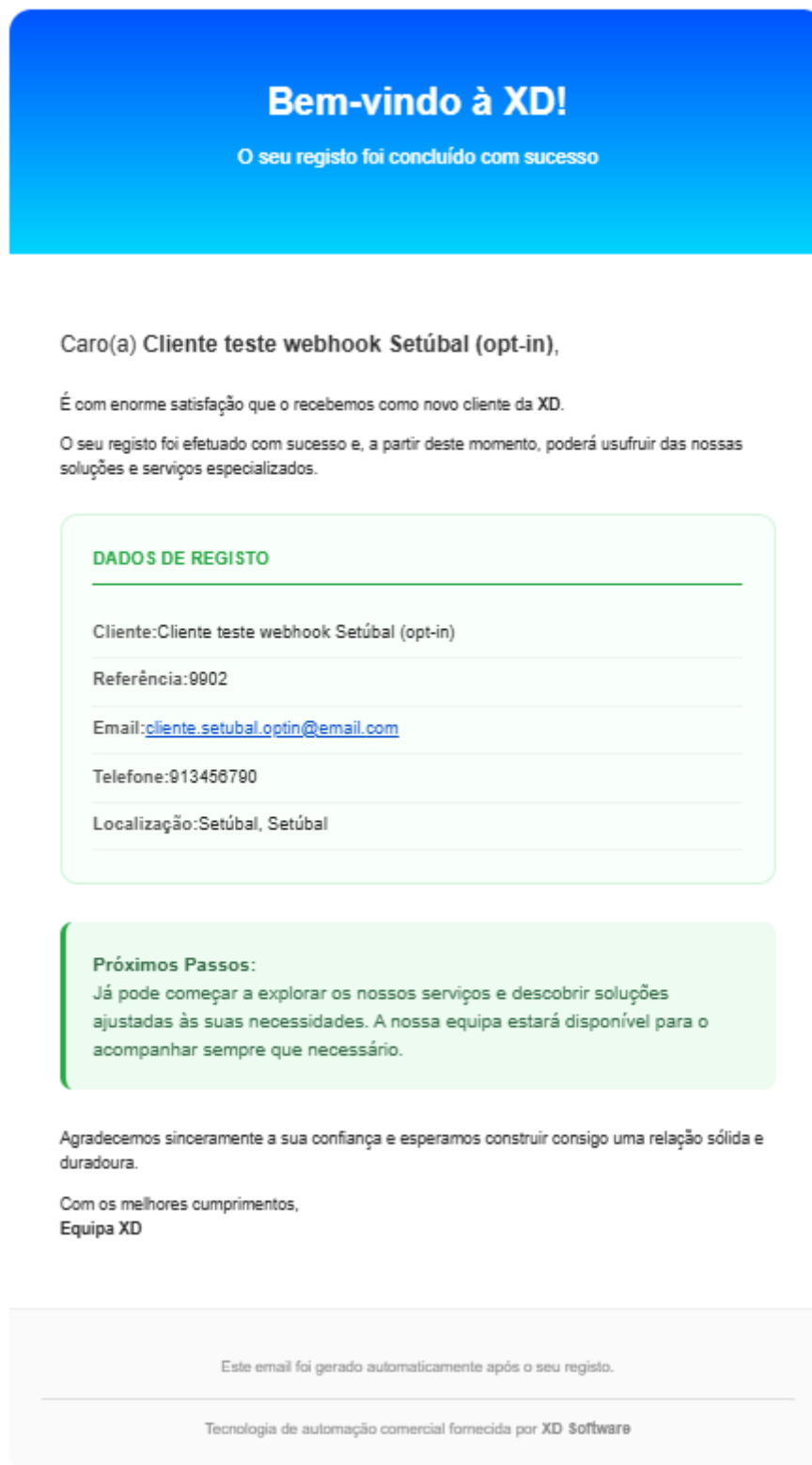


Figura 23 - E-mail boas-vindas

Whatsapp

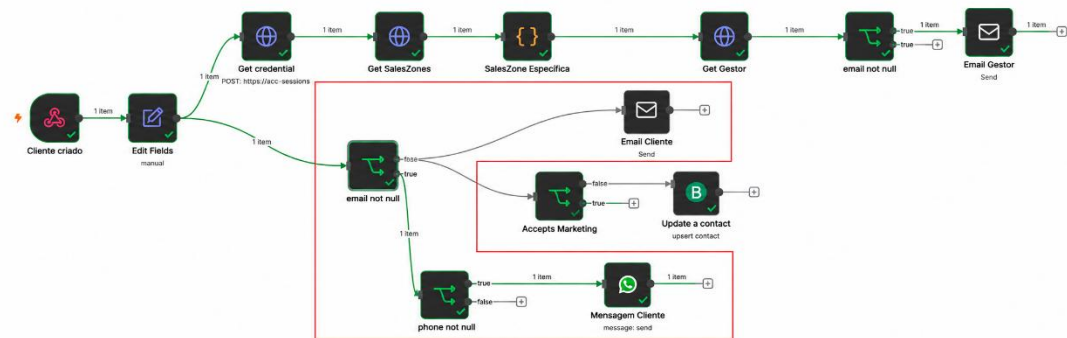


Figura 24 - UC-02: fluxo n8n whatsapp

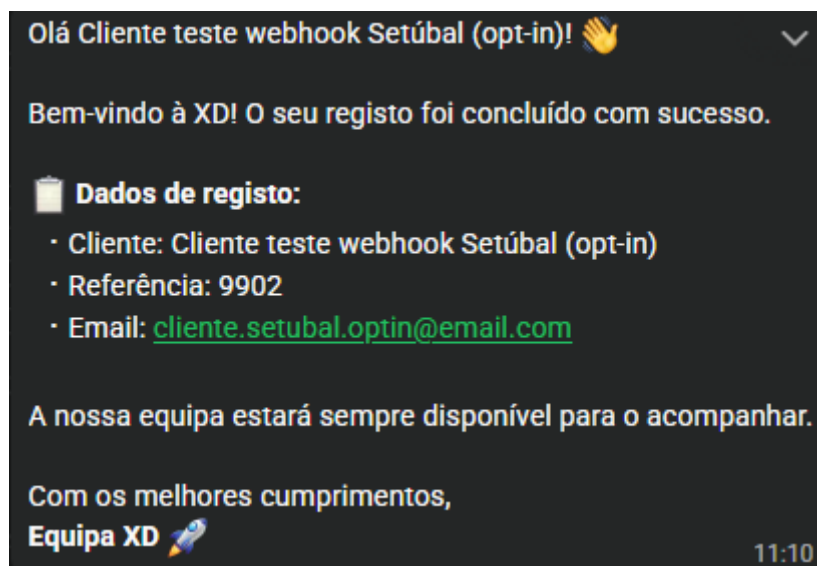


Figura 25 - Whatsapp boas-vindas

Nenhum parâmetro

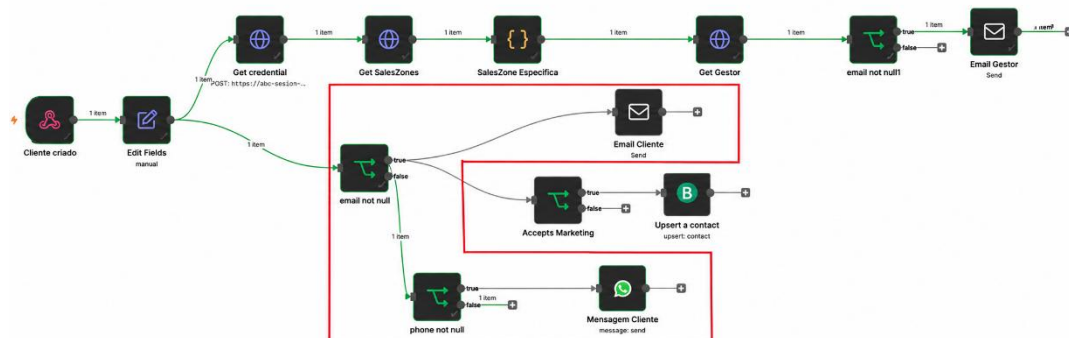


Figura 26 - UC-02: fluxo n8n vazio

4.1.3.3 UC-03: Alertar o gestor sobre clientes com inatividade comercial

Deteção e avaliação

Ao contrário dos casos de uso anteriores, este fluxo é desencadeado pelo *PeriodicWebhookWorker* e não por uma alteração na base de dados. O *CustomerInactivityDetectedPeriodicWebhookHandler* delega a deteção ao *CustomerInactiveDetectionQuery*, que avalia a inatividade de cada cliente segundo o algoritmo multicritério descrito na secção 4.1.2.2.

O gestor responsável é resolvido pela mesma normalização de distrito aplicada no UC-01, mas executada diretamente no *backend*. O *payload* resultante percorre o mesmo pipeline de entrega do fluxo reativo até ao n8n.

Orquestração no n8n

O *workflow* recebe o *array inactiveCustomers* e itera sobre cada cliente individualmente. Para cada um, verifica se o gestor possui e-mail e, em caso afirmativo, envia um alerta com os dados do cliente e indicação de necessidade de ação comercial, como é possível ser visualizado na figura 28, enquanto na figura 36 está representado o fluxo funcional no n8n desta UC-03.

Após cada envio bem-sucedido, efetua um POST */api/v1/periodic/inactivity-ack* ao *backend* para confirmar a notificação e registar o cliente no *ledger* de inatividade.



Figura 27 - UC-03: fluxo n8n

Cliente Inativo Detetado

Foi identificado um cliente com potencial risco comercial

Olá,
foi identificado um cliente que ultrapassou o período previsto de inatividade com base no seu histórico de compras.

DADOS DO CLIENTE

Nome: Francisco Rebimbas, Unip. Lda.

Identificador: 270

Email: alexia82@email.com

Telefone: 257080879

Cidade: Setúbal

Distrito: Setúbal

Última Compra: 2025-12-09

Dias sem Atividade: 162

Threshold Calculado: 38 dias

GESTOR RESPONSÁVEL

Nome: Pedro Sousa

Email: pedro.sousa@email.com

Ação Recomendada:

Recomenda-se o contacto comercial com este cliente para avaliação de possíveis ações de reativação, reforço de relacionamento e prevenção de perda de valor comercial.

A atuação célere poderá contribuir para a recuperação da atividade comercial e fortalecimento da relação com o cliente.

Figura 28 - Mensagem inatividade

Contratempos e soluções

Durante o desenvolvimento surgiu o contratempo de garantir que o mesmo cliente não recebesse alertas duplicados em caso de reinício do serviço ou execuções sobrepostas, resolvido com o *RedisCustomerInactivityNotificationLedger*, que regista os clientes notificados, e com o mecanismo de ACK que só confirma o registo após o n8n confirmar o envio.

4.1.3.4 UC-04: Sincronizar o novo cliente com a plataforma de marketing Brevo

Deteção e publicação do evento

À semelhança dos UC-01 e UC-02, este fluxo é acionado pelo mesmo evento *customer.registered*. O campo *AcceptsMarketing* incluído no *payload*, derivado do *EntityConsentmentStatus* pelo *interceptor*, é o elemento determinante para a execução deste caso de uso.

Orquestração no n8n

O *workflow* avalia se *AcceptsMarketing* está ativo. Em caso afirmativo, efetua um *upsert* na plataforma Brevo (representado na figura 29) via o nó nativo *n8n-nodes-base.sendInBlue*, criando ou atualizando o contacto na lista de contactos da figura 30. Esta abordagem garante que o mesmo evento possa ser processado múltiplas vezes sem criar duplicados. Caso *AcceptsMarketing* seja false, o fluxo termina sem sincronização.

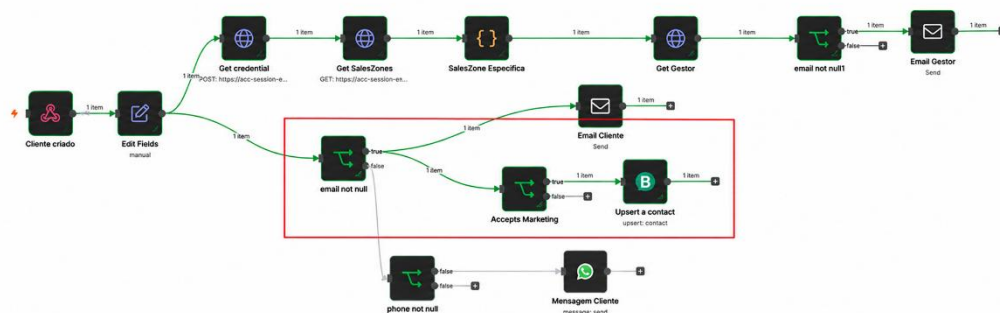


Figura 29 - UC-04: fluxo n8n

CONTATO	INSCRITO	COLOCADO EM LISTA BLOQUEADA	E-MAIL	LANDLINE_NUMBER	ÚLTIMA ALTERAÇÃO	DATA DE CRIAÇÃO
☐ 1231098@sepa.ipo.pt.com			1231098@sepa.ipo.pt.com		20/05/2026	20/05/2026
☐ cliente.schubel.opling@gmail.com			cliente.schubel.opling@gmail.com		18/05/2026	18/05/2026
☐ Bruno Gomes			bruno.gomes@et.pt		18/05/2026	18/05/2026

Figura 30 - Lista de clientes Brevo

Contratempos e soluções

A principal decisão de implementação foi definir onde e como derivar o consentimento de marketing. O campo *AcceptsMarketing* não existe nativamente no ERP, foi introduzido pelo *interceptor* com base no *EntityConsentmentStatus* do cliente.

Garantir que este campo chegasse ao n8n com o valor correto implicou enriquecer o *payload* ainda antes da publicação no Redis, criando uma cópia independente do *payload* para o evento *customer.registered* de forma a não interferir com o evento genérico *entity*.^{*}

4.2 Testes

A verificação da solução desenvolvida foi conduzida através de uma suite de testes automatizados que cobre os principais componentes do módulo de Webhooks, desde a lógica de avaliação de negócio até às fronteiras da API e aos serviços de execução em segundo plano. A organização dos testes reflete a arquitetura da solução, partindo dos componentes mais internos em direção às interfaces externas, garantindo que cada camada é validada de forma independente antes de ser integrada no fluxo completo.

4.2.1 Abordagem de verificação

A verificação da solução seguiu uma estratégia em camadas, partindo dos componentes mais internos em direção às fronteiras do sistema. Esta abordagem garante que cada unidade de lógica é validada de forma isolada antes de ser integrada no fluxo completo, tornando mais fácil identificar a origem de eventuais falhas.

O ecossistema de testes utilizado assenta em três ferramentas complementares: NUnit como *framework* de execução e organização dos testes, *FluentAssertions* para asserções legíveis e expressivas que tornam o propósito de cada verificação imediatamente

claro, e Moq para a substituição de dependências externas por substitutos controlados (*mocks*), permitindo o isolamento total de cada componente testado sem necessidade de infraestrutura real como Redis, base de dados ou serviços HTTP.

Os testes foram organizados em torno dos dois fluxos principais da arquitetura, reativo e periódico, refletindo a separação natural do sistema.

- O fluxo **reativo** é coberto pelos testes ao pipeline de eventos, ao *interceptor* e ao *worker* de entrega.
- O fluxo **periódico** é coberto pelos testes ao *worker* periódico, ao plugin de agendamento semanal e ao *handler* de detecção de inatividade.
- Complementarmente, os testes às **interfaces de comunicação** validam as fronteiras da API e o comportamento face a condições de erro.

Esta organização permitiu que a suite de testes crescesse de forma incremental ao longo do desenvolvimento, adicionando cobertura à medida que cada componente era implementado, e que a integração contínua na plataforma Gitea executasse os testes automaticamente em cada alteração ao repositório.

4.2.2 Verificação dos componentes de domínio e aplicação

O *CustomerInactivityEvaluator* constitui o componente com maior densidade de testes dado o número de variáveis que governa o seu comportamento.

Os cinco testes implementados percorrem os três ramos de decisão do algoritmo:

- o ramo para clientes sem qualquer compra registada, onde o limiar é calculado com base nos dias desde o registo;
- o ramo para clientes com histórico insuficiente para cálculo estatístico, onde se aplica o *MinimumInactiveDays* como piso;
- e o ramo com histórico suficiente, onde o limiar dinâmico resulta da mediana dos intervalos entre compras multiplicada por *AverageGapMultiplier*.

Valida-se ainda que a fusão entre a data de última compra da entidade e as datas dos documentos determina corretamente o valor mais recente. A Listagem 6 ilustra a estrutura destes testes.

Listagem 6 - Classe *CustomerInactivityEvaluatorTests*

```

public sealed class CustomerInactivityEvaluatorTests
{
    [Test]
     Daniel Novo
    public void Never_purchased_inactive_after_max_of_never_purchased_rule_and_min_quiet_days() {...}

    [Test]
     Daniel Novo
    public void Never_purchased_not_inactive_before_threshold() {...}

    [Test]
     Daniel Novo
    public void With_purchases_uses_max_of_history_threshold_and_min_quiet_days() {...}

    [Test]
     Daniel Novo
    public void Median_with_even_number_of_gaps_uses_average_of_two_central_gaps() {...}

    [Test]
     Daniel Novo
    public void Last_purchase_on_entity_merged_with_document_dates_for_last_activity_day() {...}
}

```

O *CustomerInactivityDetectedPeriodicWebhookHandler* é testado em dois cenários complementares: ausência de clientes inativos, onde o resultado deve impedir a publicação; e presença de clientes não notificados, onde o *payload* gerado deve conter os dados corretos e as linhas a registrar no *ledger* devem corresponder aos clientes identificados, conforme a Listagem 7.

Listagem 7 - Classe *CustomerInactivityDetectedPeriodicWebhookHandlerTests*

```

public class CustomerInactivityDetectedPeriodicWebhookHandlerTests
{
    [Test]
    public async Task EvaluateAsync_returns_false_when_no_inactive_customers_found() ...

    [Test]
    public async Task EvaluateAsync_returns_payload_and_rows_to_commit_when_new_inactive_customers_exist() ...

    private static IOptionMonitor<PeriodicWorkerOptions> BuildOptions() ...

    private static PeriodicWebhookEventDefinition BuildDispatchContext() => ...;
}

```

O *WebhookEventPipeline* é coberto por testes que verificam o comportamento face a entradas inválidas (JSON malformatado e *tenant* em falta são rejeitados sem consultar o repositório) e face a entradas válidas, onde se confirma o enfileiramento de uma tarefa por

subscrição e a aplicação do transformador de *payload* adequado ao tipo de evento, como demonstrado na Listagem 8.

Listagem 8 - Classe WebhookEventPipelineTests

```
public class WebhookEventPipelineTests
{
    private Mock<IWebhookSubscriptionRepository> _subscriptionRepository = null!;
    private Mock<IWebhookLoggingService> _loggingService = null!;
    private Mock<IWebhookMetricsService> _metricsService = null!;
    private Mock<IWebhookEventDispatcher> _dispatcher = null!;
    private Mock<IWebhookDeliverySink> _deliverySink = null!;
    private Mock<IItemsCreatedInventoryPayloadTransformer> _inventoryTransformer = null!;
    private Mock<IPurchaseCompletedPayloadTransformer> _purchaseTransformer = null!;
    private WebhookEventPipeline _sut = null!;

    [SetUp]
    public void Setup() { ... }

    [Test]
    public async Task ProcessAsync_returns_false_for_invalid_json() { ... }

    [Test]
    public async Task ProcessAsync_returns_false_when_tenant_missing_and_channel_invalid() { ... }

    [Test]
    public async Task ProcessAsync_returns_true_and_queues_when_subscriptions_exist() { ... }

    [Test]
    public async Task ProcessAsync_returns_true_when_no_subscriptions() { ... }

    [Test]
    public async Task ProcessAsync_applies_purchase_completed_transformer_when_items_transformer_returns_null() { ... }
}
```

4.2.3 Verificação dos fluxos de execução em segundo plano

O *WeeklyPeriodicScheduleModePlugin* é validado por dois testes que cobrem as responsabilidades do plugin:

- o *IsDue* deve retornar *false* antes da hora configurada e *true* a partir do momento exato, respeitando o *TimeZoneId* da subscrição;
- o *TryPrepareDispatchAsync* deve invocar o verificador Redis com o *slot* correto, a data do dia agendado na semana corrente, garantindo que a mesma subscrição não dispare mais do que uma vez por semana, tal como ilustrado na Listagem 9.

Listagem 9 - Classe WeeklyPeriodicScheduleModePluginTests

```
public class WeeklyPeriodicScheduleModePluginTests
{
    private readonly WeeklyPeriodicScheduleModePlugin _sut = new();

    [Test]
    public void IsDue_returns_true_only_after_scheduled_weekday_and_time() {...}

    [Test]
    public async Task TryPrepareDispatchAsync_calls_verifier_with_week_slot_date() {...}
}
```

O *PeriodicWebhookWorker* é testado através da invocação direta do método interno *RunTickAsync*, verificando que a cadeia completa (avaliação de *schedule*, preparação do *slot*, avaliação de negócio e publicação no Redis) é executada na sequência correta quando todas as condições estão satisfeitas, como evidenciado na Listagem 10.

Listagem 10 - Classe PeriodicWebhookWorkerTests

```
public class PeriodicWebhookWorkerTests
{
    [Test]
    public async Task RunTickAsync_publishes_event_when_definition_is_due_and_handler_allows() {...}
}
```

O *WebhookDeliveryWorker* é testado num cenário de ciclo completo: uma tarefa é enfileirada após o arranque do serviço, o *worker* processa-a, invoca o serviço de entrega HTTP e regista a tentativa no repositório de *logs*, confirmando o funcionamento do mecanismo assíncrono de ponta a ponta, conforme a Listagem 11.

Listagem 11 - Classe WebhookDeliveryWorkerTests

```
public class WebhookDeliveryWorkerTests
{
    [Test]
    public async Task Worker_processes_enqueued_delivery_task() {...}
}
```

4.2.4 Verificação da camada de interface

Os testes ao *WebhookSubscriptionsController* cobrem as condições de fronteira da API:

- pedidos sem *tenant* autenticado retornam 401 *Unauthorized*;
- criação com dados inválidos retorna 400 *BadRequest*;
- eliminação de recurso inexistente retorna 404 *NotFound*.

. A Listagem 12 demonstra estes cenários.

Listagem 12 - Classe *WebhookSubscriptionsControllerTests*

```
public class WebhookSubscriptionsControllerTests
{
    private Mock<IWebhookSubscriptionService> _subscriptionService = null!;
    private Mock<IWebhookEventCatalogService> _eventCatalogService = null!;
    private Mock<ITenantService> _tenantService = null!;
    private WebhookSubscriptionsController _sut = null;

    [SetUp]
    public void SetUp() { ... }

    [Test]
    public async Task GetAvailableEventTypes_returns_unauthorized_when_tenant_is_missing() { ... }

    [Test]
    public async Task GetAll_returns_ok_with_subscriptions() { ... }

    [Test]
    public async Task Create_returns_bad_request_when_service_throws_argument_exception() { ... }

    [Test]
    public async Task Delete_returns_not_found_when_service_throws_key_not_found() { ... }

    private void SetUserTenantClaim(string tenantId) { ... }
}
```

Numa fase anterior à integração com o ERP em ambiente real, o *LocalWebhookEventsController* foi utilizado para injetar eventos diretamente no canal Redis, permitindo validar os *workflows* n8n de forma independente e confirmar que os *payloads* eram processados corretamente pela plataforma de orquestração.

4.2.5 Resultados e integração contínua

O repositório de código está integrado com a plataforma Gitea, onde a pipeline de CI/CD executa automaticamente a suite de testes em cada *push* ou *pull request*, garantindo que alterações ao código não introduzem regressões nos componentes críticos da solução.

A Listagem 13 apresenta o resultado da execução completa, confirmando que todos os cenários implementados foram validados com sucesso.

Listagem 13 - Run testes (succeeded)

```
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in C:\Users\danie\RiderProjects\XDSoba\XDPeople.Soba.Webhooks\XDPeople.Soba.Webhooks.Tests\
  NUnit3TestExecutor discovered 155 of 155 NUnit test cases using Current Discovery mode, Non-Explicit run
NUnit Adapter 4.5.0.0: Test execution complete
  XDPeople.Soba.Webhooks.Tests test net8.0 succeeded (1,3s)

Test summary: total: 155; failed: 0; succeeded: 155; skipped: 0; duration: 1,3s
Build succeeded with 32 warning(s) in 3,3s
```

4.3 Avaliação da solução

A avaliação da solução foi conduzida por confronto direto entre os requisitos definidos na secção 3.2 e os resultados obtidos na implementação, recorrendo às evidências recolhidas ao longo do capítulo anterior.

4.3.1 Cumprimento dos requisitos funcionais

A Tabela 6 apresenta o grau de realização de cada requisito funcional, com referência às evidências que comprovam a sua concretização.

Tabela 6 - Avaliação dos requisitos funcionais

ID	Descrição	Estado	Evidência
RF-01	Notificar gestor sobre novo cliente	Concretizado	Figuras 29 e 30
RF-02	Boas-vindas ao novo cliente (e-mail ou WhatsApp)	Concretizado	Figuras 31 a 35
RF-03	Alertar gestor sobre cliente inativo	Concretizado	Figuras 36 e 37
RF-04	Sincronização com Brevo	Concretizado	Figuras 38 e 39

Todos os requisitos funcionais foram concretizados na íntegra. O RF-02 merece destaque pela lógica de *fallback* implementada, quando o cliente não possui endereço de e-mail, o sistema recorre automaticamente ao WhatsApp, garantindo que a ausência de um canal não impede a comunicação.

4.3.2 Cumprimento dos requisitos não funcionais

A Tabela 7 apresenta uma síntese do grau de cumprimento de cada requisito não funcional, detalhada nos parágrafos seguintes.

Tabela 7 - Avaliação dos requisitos não funcionais

ID	Categoria	Grau de cumprimento
RNF-01	Usabilidade	Concretizado
RNF-02	Fiabilidade	Concretizado
RNF-03	Desempenho	Concretizado (não medido quantitativamente)
RNF-04	Suportabilidade	Concretizado
RNF-05	Segurança	Concretizado
RNF-06	Design	Concretizado

- **RNF-01 — Usabilidade:** Os *workflows* foram desenvolvidos e configurados na plataforma n8n, sendo este o canal principal de implementação das automações de negócio. Ao longo do desenvolvimento, foram também necessárias iterações ao *backend* para ajustar o formato dos *payloads*, o comportamento do pipeline, a configuração dos eventos periódicos, entre outros, o que é expectável numa fase inicial de integração entre sistemas. O sistema tolera corretamente a ausência de dados opcionais, clientes sem e-mail nem telefone terminam o fluxo de forma controlada sem produzir erros, conforme demonstrado na Figura 35.
- **RNF-02 — Fiabilidade:** A política de reenvio com atrasos progressivos de [0, 30, 300] segundos e até 3 tentativas, implementada no *WebhookDeliveryWorker*, garante resiliência face a falhas transitórias do n8n. O *RedisCustomerInactivityNotificationLedger*,

combinado com o mecanismo de ACK, assegura que cada cliente inativo é notificado exatamente uma vez, mesmo em caso de reinício do serviço.

- **RNF-03 — Desempenho:** A publicação no Redis é realizada numa *Task.Run* independente após o *SavedChangesAsync*, garantindo que o pedido HTTP do ERP retorna sem aguardar a cadeia de entrega. Durante os testes funcionais não foi registado qualquer atraso perceptível no fluxo de registo de clientes no ERP com o pipeline de eventos ativo. Não foram realizadas medições quantitativas formais de latência, pelo que este requisito é considerado concretizado a nível arquitetural e observacional.
- **RNF-04 — Suportabilidade:** A arquitetura de *plugins* (*IPeriodicScheduleModePlugin*) permite adicionar novos modos de agendamento sem alterar o *PeriodicWebhookWorker*. A separação entre subscrições e lógica de entrega permite incorporar novos tipos de evento e novos destinos sem redesenhar a solução existente.
- **RNF-05 — Segurança:** A autenticação nos *workflows* n8n é realizada por *tokens* JWT obtidos por pedido a cada execução. A comunicação entre o *backend* e o n8n é protegida por assinatura digital HMAC incluída no cabeçalho de cada entrega, validada pelo *endpoint receptor*.
- **RNF-06 — Design:** A solução foi desenhada para que o *backend* e o n8n comuniquem exclusivamente através de eventos *webhook*, mantendo as duas camadas desacopladas ao nível da lógica de negócio. Na prática, o desenvolvimento implicou ajustes iterativos em ambas as camadas em simultâneo, algo inerente ao processo de integração de sistemas novos. O desacoplamento efetivo verifica-se ao nível estrutural, visto que alterações aos *workflows* n8n não exigem recompilação nem *redploy* do *backend*, e vice-versa.

4.3.3 Discussão

A solução concretizou a totalidade dos requisitos definidos, com os graus de cumprimento esperados para um projeto desta natureza.

Os requisitos funcionais foram todos implementados e validados com evidências reais (notificações entregues, mensagens recebidas e contactos sincronizados) demonstrando que o sistema responde corretamente aos eventos.

Ao nível dos requisitos não funcionais, o desempenho constitui o ponto com menor evidência formal: a garantia de baixa latência é arquitetural e observacional, mas não suportada por medições quantitativas.

O processo de desenvolvimento revelou que o desacoplamento total entre *backend* e plataforma de orquestração (n8n) é um objetivo de maturidade progressiva, na fase inicial de integração, são esperados ajustes em ambas as camadas em simultâneo. À medida que os contratos de *payload* estabilizam, a independência entre as camadas torna-se efetiva, o que se verificou na fase final do projeto.

Uma limitação transversal identificada é a dependência de serviços externos para o envio de comunicações, e-mail, WhatsApp e a sincronização de contactos com a plataforma Brevo. A política de *retry* mitiga falhas transitórias, mas falhas prolongadas resultariam em notificações não entregues sem mecanismo de recuperação automática para além das 3 tentativas configuradas. A implementação de uma fila persistente de entregas falhadas seria uma melhoria relevante.

5 Conclusões

Ao longo do projeto foi desenvolvida uma solução integrada no contexto do XD Fusion, orientada para a automação de processos empresariais através de uma arquitetura *event-driven*, com especial incidência na gestão automatizada do relacionamento com clientes.

O trabalho partiu da necessidade de reduzir a dependência de processos manuais em sistemas ERP e POS, sobretudo em situações em que a informação registada no sistema exige uma resposta rápida, como o registo de novos clientes ou a identificação de clientes sem atividade comercial recente. Neste sentido, a solução desenvolvida permitiu evoluir de um modelo baseado em consulta manual de dados para um modelo mais reativo, no qual determinados eventos de negócio desencadeiam automaticamente fluxos de processamento e comunicação.

A solução implementada recorreu a *webhooks*, integração com API REST, autenticação por JWT, Redis como mecanismo de suporte à publicação e distribuição de eventos, e n8n como plataforma de orquestração de *workflows*. Esta combinação permitiu criar uma arquitetura flexível, modular e preparada para a integração com diferentes serviços externos, nomeadamente canais de comunicação e plataformas de marketing digital.

5.1 Objetivos concretizados

Esta secção avalia em que medida cada um dos objetivos propostos foi alcançado. A Tabela 7 fornece um resumo do estado de cada objetivo e as respetivas observações.

Tabela 8 - Matriz de análise da concretização dos objetivos pretendidos

Objetivo	Estado	Observações
Desenvolvimento de <i>templates</i> de automação	Concretizado	Foram implementados quatro <i>workflows</i> n8n (UC-01 a UC-04) cobrindo cenários reativos e periódicos de negócio.
Integração com a API REST do ERP/POS via JWT	Concretizado	A autenticação JWT e o consumo de endpoints foram implementados, permitindo acesso a dados de clientes, zonas de venda e gestores em tempo real.
Adoção de arquitetura <i>event-driven</i>	Concretizado	Implementados fluxo reativo (Redis Pub/Sub + <i>WebhookDeliveryWorker</i>) e fluxo periódico (<i>PeriodicWebhookWorker</i>), conforme detalhado no Capítulo 4.
Automação de processos empresariais	Concretizado	Notificações, boas-vindas, alertas de inatividade e sincronização com Brevo foram totalmente automatizados via n8n
Testes e validação da solução	Concretizado	Testes unitários, de integração e funcionais implementados e integrados na pipeline de CI/CD, todos executados com sucesso.
Documentação técnica	Concretizado	Arquitetura, casos de uso, fluxos e mecanismos de validação documentados ao longo do presente relatório.

O primeiro objetivo proposto consistia no desenvolvimento de *templates* de automação que executassem ações em resposta a eventos do sistema. Foi concretizado com a criação de quatro *workflows* em n8n, cada um associado a um caso de uso específico: notificação do gestor comercial perante o registo de um novo cliente, envio de mensagem de boas-vindas ao cliente por e-mail ou WhatsApp, alerta ao gestor sobre clientes com inatividade comercial e sincronização de contactos com a plataforma Brevo.

O segundo objetivo dizia respeito à integração com a API REST do ERP/POS, com autenticação por JWT e consumo dos *endpoints* necessários. A sua concretização permitiu que

os *workflows* acessem em tempo real a informação operacional relevante, como dados de clientes, zonas de venda e gestores responsáveis.

O terceiro objetivo consistia na adoção de uma arquitetura *event-driven*. Este foi o pilar estrutural de todo o projeto, concretizado através da implementação de um pipeline de publicação, subscrição e entrega de eventos. Foram suportados dois modos de funcionamento: um fluxo reativo, acionado por eventos em tempo real como o *customer.registered*, e um fluxo periódico, responsável pela deteção e notificação de inatividade comercial com base em critérios previamente definidos.

O quarto objetivo centrava-se na automação de processos empresariais repetitivos. A implementação dos quatro casos de uso demonstrou, em contexto real, que a arquitetura desenvolvida é capaz de substituir tarefas manuais por respostas automáticas, consistentes e imediatas. As notificações foram entregues, as mensagens de boas-vindas enviadas e os contactos sincronizados, validando o funcionamento *end-to-end* da solução.

O quinto objetivo abrangia os testes e a validação da solução. Foram desenvolvidos testes para os principais componentes, domínio, aplicação, *workers* de segundo plano, mecanismos de entrega e camada de interface, integrados numa pipeline de integração contínua que executa automaticamente a suite a cada alteração ao repositório. Todos os testes foram concluídos com sucesso.

Por fim, o objetivo de documentação técnica foi igualmente cumprido através do presente relatório, que descreve em detalhe a arquitetura, os casos de uso, os fluxos implementados e os resultados obtidos, constituindo um suporte para a manutenção e evolução futura da plataforma.

Em suma, os objetivos definidos foram concretizados na totalidade. A única ressalva a registar prende-se com a validação de desempenho, que se manteve ao nível arquitetural e observacional, sem medições quantitativas formais.

5.2 Limitações e trabalho futuro

A limitação mais imediata identificada é a dependência de serviços externos — e-mail, WhatsApp e Brevo — para a execução de parte dos fluxos de automação. A política de retry implementada confere resiliência face a falhas transitórias, mas não cobre cenários de indisponibilidade prolongada. A criação de uma fila persistente de entregas falhadas seria uma evolução natural, garantindo que nenhuma notificação se perde definitivamente por falha temporária de um serviço terceiro.

A ausência de métricas quantitativas de desempenho constitui outra limitação relevante. Embora a arquitetura assíncrona minimize o impacto no fluxo normal do ERP, não foram realizados testes de carga nem definidos indicadores formais de latência ou *throughput*. A instrumentação da solução com ferramentas de observabilidade permitiria monitorizar o comportamento em produção e sustentar decisões de escalabilidade com base em dados concretos.

O desacoplamento entre o *backend* e o *frontend*, embora estruturalmente garantido, revelou-se progressivo na prática. A fase inicial de integração implicou ajustes iterativos em ambas as camadas em simultâneo. À medida que os contratos de *payload* estabilizaram, a independência tornou-se efetiva. Para consolidar este ganho, recomenda-se a formalização dos contratos de evento através de esquemas de validação ou documentação técnica versionada.

Do ponto de vista de evolução funcional, a arquitetura está preparada para suportar novos tipos de evento sem alterações estruturais. A extensão para cenários como emissão de faturas, alterações de stock, pagamentos pendentes ou mudanças de estado de encomendas representaria um alargamento significativo do valor da plataforma XD Fusion. Por fim, a implementação de uma interface de monitorização e auditoria que permita acompanhar o estado dos eventos, entregas e workflows, tornaria o sistema mais transparente e operacionalmente mais robusto.

5.3 Apreciação final

O estágio curricular na XD Software constituiu uma experiência de elevado valor formativo, proporcionando a aplicação de conhecimentos adquiridos ao longo da licenciatura num ambiente profissional real, com requisitos concretos e integração com sistemas em produção.

O contacto com áreas como arquiteturas orientadas a eventos, processamento assíncrono, integração de sistemas e automação de processos permitiu consolidar competências técnicas relevantes para o exercício da engenharia informática.

Do ponto de vista técnico, o projeto evidenciou que o desacoplamento entre componentes e a modularidade da arquitetura são fatores determinantes para a sustentabilidade e extensibilidade de uma solução. A substituição de processos manuais por respostas automáticas a eventos demonstrou ser uma abordagem eficaz para aumentar a eficiência operacional e reduzir a margem de erro humano.

A nível pessoal, o estágio reforçou a importância de compreender o domínio de negócio antes de tomar decisões técnicas, bem como o papel crítico da documentação e dos testes na construção de software fiável. A colaboração com a equipa da XD Software e o acompanhamento do orientador foram determinantes para a qualidade do trabalho produzido.

Conclui-se que a solução desenvolvida cumpre os objetivos propostos e representa um contributo concreto para a plataforma XD Fusion. As limitações identificadas não comprometem a validade da abordagem adotada e apontam para direções de evolução claras. A base técnica construída durante o estágio está preparada para suportar a expansão progressiva da arquitetura *event-driven* no ecossistema da XD Software.

Referências

- Arham Ehsan. (2025). *Understanding FURPS+ Requirements Model | PDF | Usability | Reliability Engineering*. <https://pt.scribd.com/document/509825759/FUPRS>
- codesee. (2025). *C4 Model: Importance, Use Cases, and Examples*. <https://www.codesee.io/learning-center/c4-model>
- Hohpe, G. (2006). *Programming Without a Call Stack-Event-driven Architectures*. www.eaipatterns.com
- Kruchten, P. (1995). Architectural Blueprints-The «4+1» View Model of Software Architecture. *IEEE Software*, 12(6), 42–50.
- Lokesh Gupta. (2025). *What is REST?: REST API Tutorial*. <https://restfulapi.net/>
- Martin Fowler. (2017). *What do you mean by “Event-Driven”?* <https://martinfowler.com/articles/201701-event-driven.html>
- Matthew Guay. (2025). *What are webhooks? | Zapier*. https://zapier.com/blog/what-are-webhooks/?utm_source=chatgpt.com
- Mozilla Developer Network. (2024). *REST - Glossary | MDN*. <https://developer.mozilla.org/en-US/docs/Glossary/REST>
- Oracle. (2025). *What Is ERP? | Oracle*. <https://www.oracle.com/erp/what-is-erp/>
- Salesforce. (2024). *What Is CRM (Customer Relationship Management)? | Salesforce*. <https://www.salesforce.com/crm/what-is-crm/>
- Steffi. (2025). *Webhooks: What are they and how are they used? - Zoho Blog*. <https://www.zoho.com/blog/assist/webhooks-what-are-they-and-how-are-they-used.html>
- Venelouis. (2024). *O que diabos é uma «API»?! E pior ainda uma «API REST(ful)»? | @venelouis | API Rest | DIO*. <https://www.dio.me/articles/o-que-diabos-e-uma-api-e-pior-ainda-uma-api-restful>
- Vockan. (2024). *ERP: O Que é e Como Implementá-lo para Otimizar Seus Processos - Vockan*. <https://vockan.com/o-que-e-erp/>